



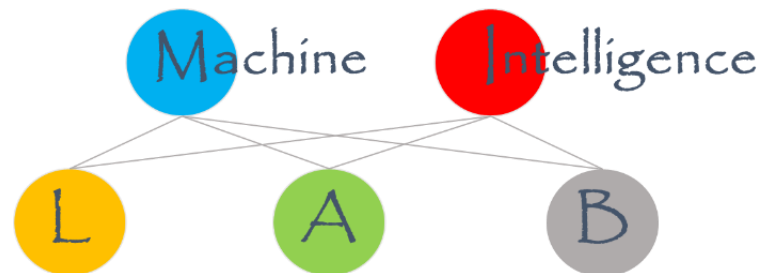
# Pixel Computing – Part 2

Mu Yadong

Machine Intelligence Lab

Institute of Computer Science & Technology

Peking University



**Most slides are adapted from related courses or tutorials.  
Internal use only. Please do not distribute the slides.**

# Outline

- **Pixel Computing**
- **Pixel-Level Methods**
  - FCN
  - SegNet
  - DeepLab
  - CNN-as-RNN
- **Instance-Level Methods**
  - Mask R-CNN
- **Human Pose Estimation**
  - Bottom-Up Methods
  - Top-Down Methods
  - 3D Pose Estimation

# Pixel Computing

- Assign labels at the pixel label (“pixel in, pixel out”)
- May or may not enforce strong structural constraints / priors

semantic segmentation



Object parsing





COCO  
Common Objects in Context

cocodataset@outlook.com

[Home](#) [People](#) [Explore](#) [Dataset](#) [External](#)

COCO 2014 train/val browser (123,287 images, 886,284 instances). Crowd labels not shown.

  
[search](#)

- Object segmentation
- Recognition in Context
- Multiple objects per image
- More than 300,000 images
- More than 2 Million instances
- 80 object categories
- 5 captions per image
- Keypoints on 100,000 people

Dataset examples



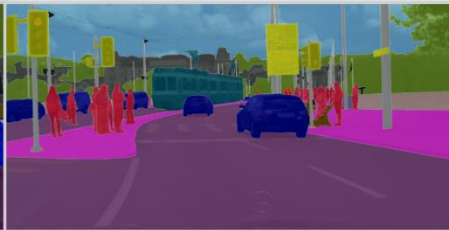


# CITYSCAPES DATASET

- Semantic Understanding of Urban Street Scenes
- Pixel-level and instance-level



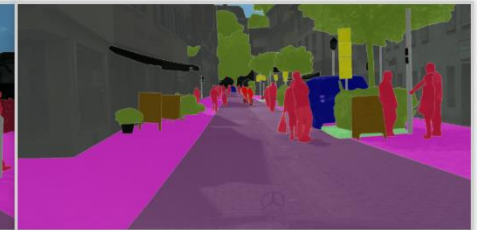
Stuttgart



Zurich



Ulm



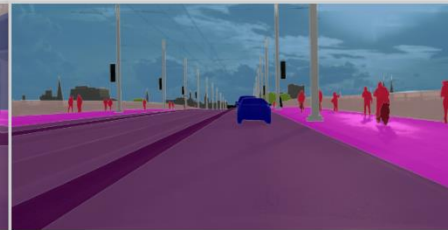
Tübingen



Münster



Cologne



Bonn



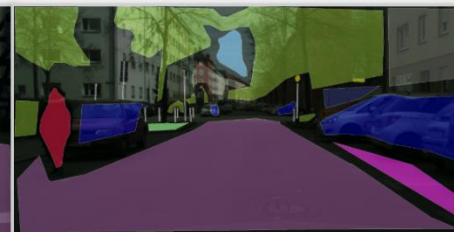
Erfurt



Saarbrücken



Saarbrücken



Nuremberg



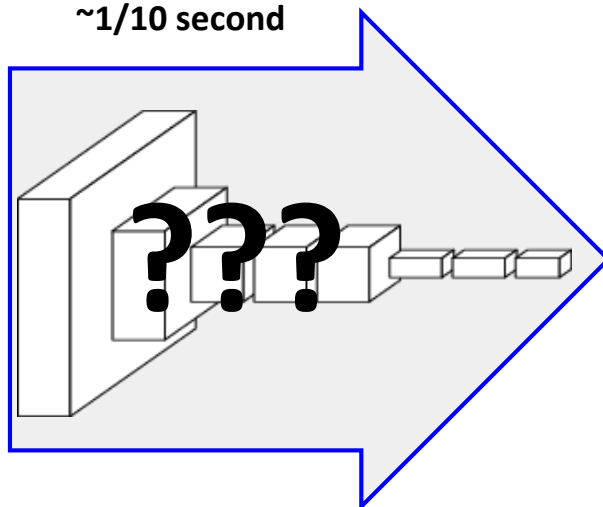
Nuremberg

<https://www.cityscapes-dataset.com/benchmarks/>

# Image Segmentation with CNN?

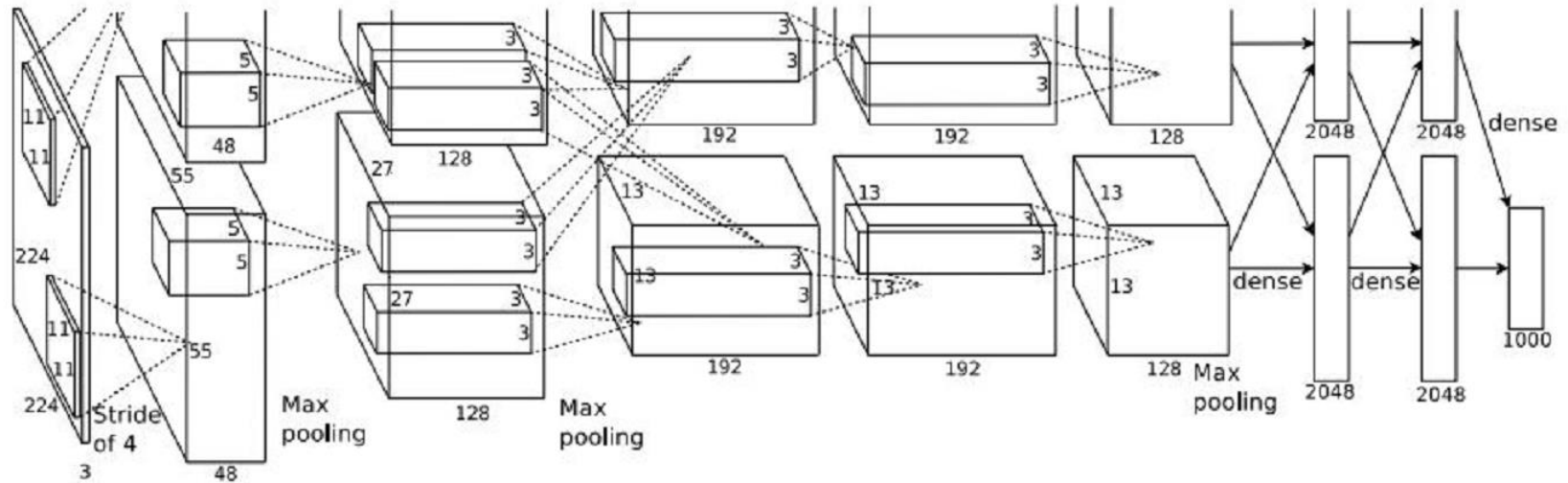


~1/10 second



end-to-end learning

# Image Segmentation with CNN?



**CNN classifiers are operating on high-level neurons. We can't tell accurate positions.**

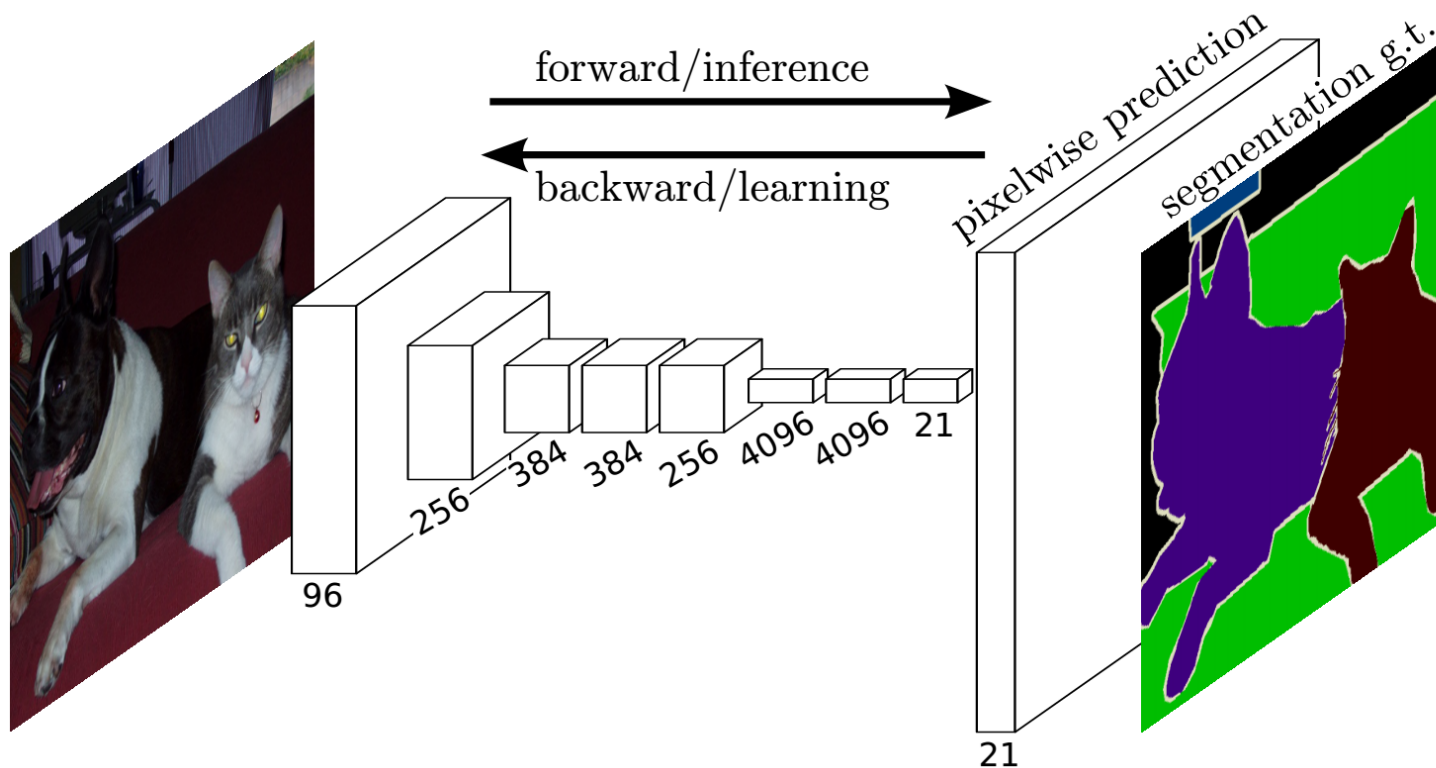
**Solutions: de-convolution, hyper-network etc.**

## Key Observations

- CNNs yield a coarse prediction on pixel labeling tasks
- Structural optimization (e.g., CRFs) improves the result by accounting for the contextual information in the image.
- Learning the whole pipeline end-to-end significantly improves the results

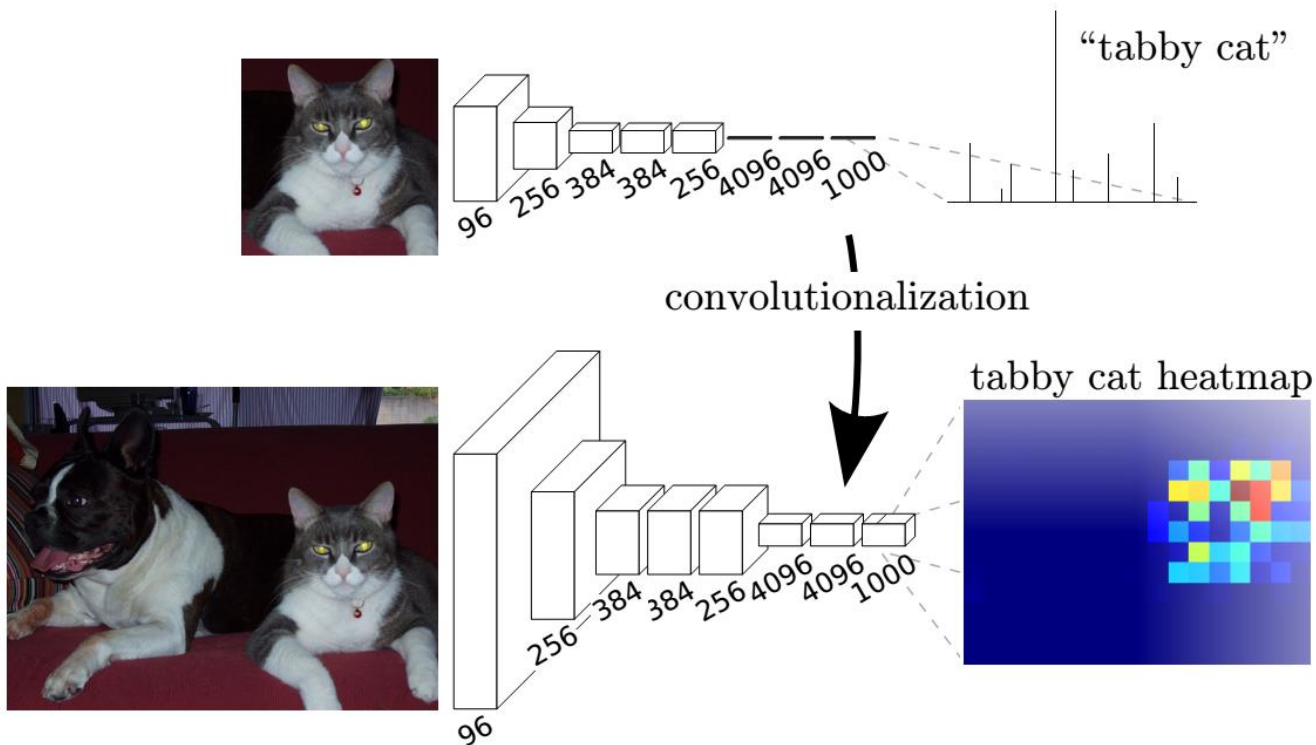
# Fully Convolutional Networks for Semantic Segmentation (CVPR 2015)

- Seminal work conducted by Jonathan Long \* Evan Shelhamer\* Trevor Darrell from U.C. Berkeley



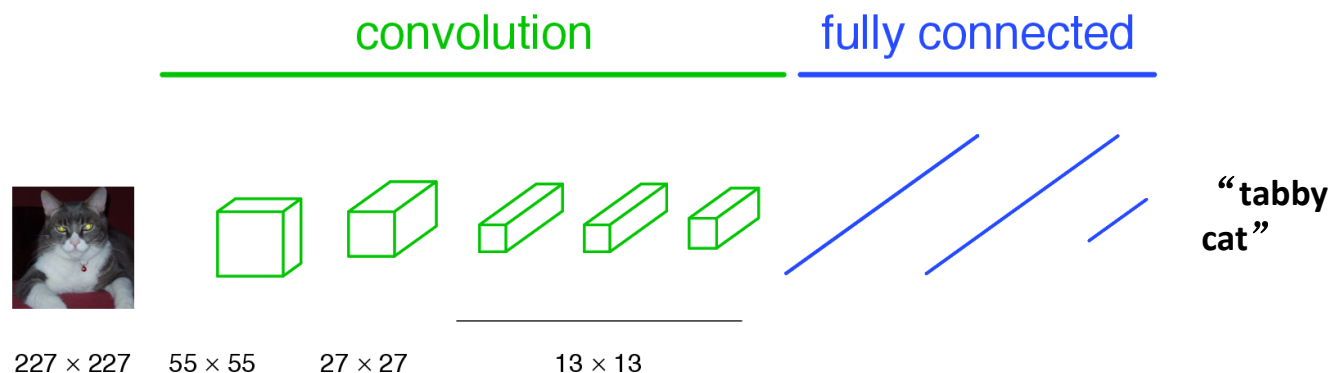
# Converting to Fully Convolutional Networks

- Each network layer is a 3-D array
- We need to re-think about the fc layers

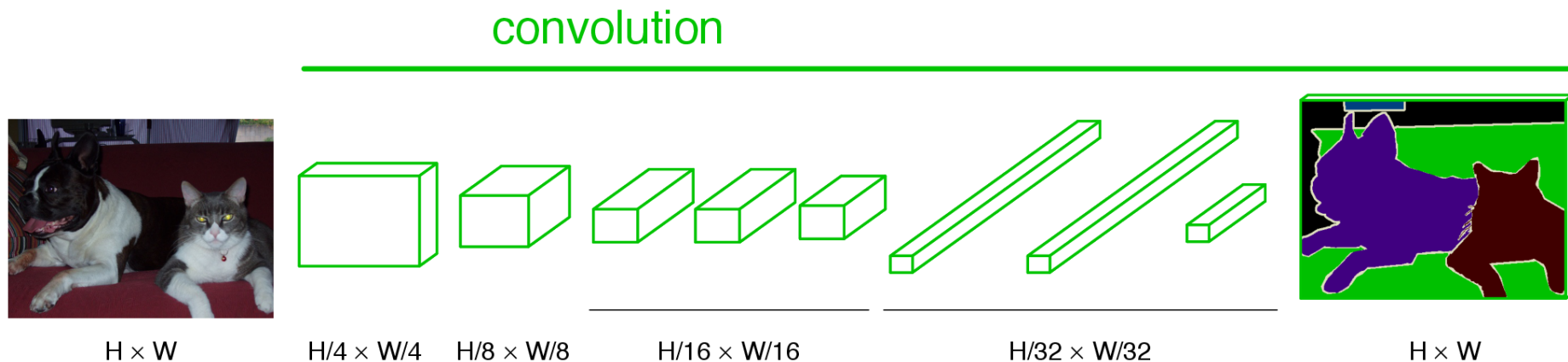


# Comparing Classification / Segmentation Pipeline

## ■ Classification



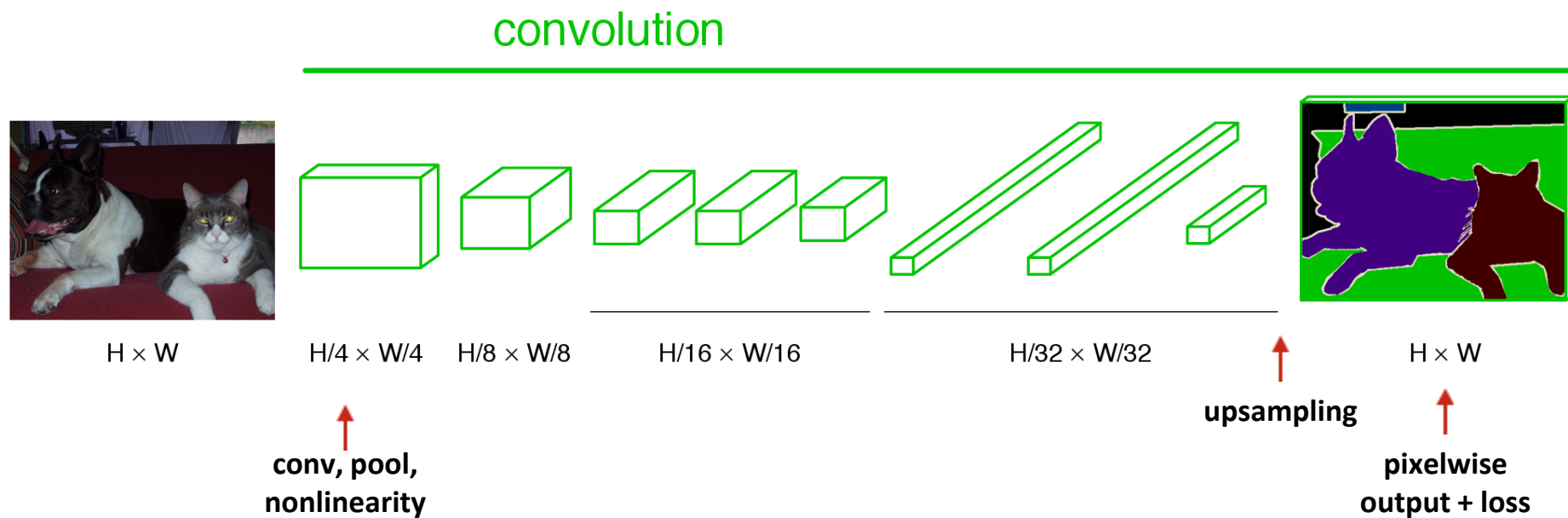
## • Segmentation





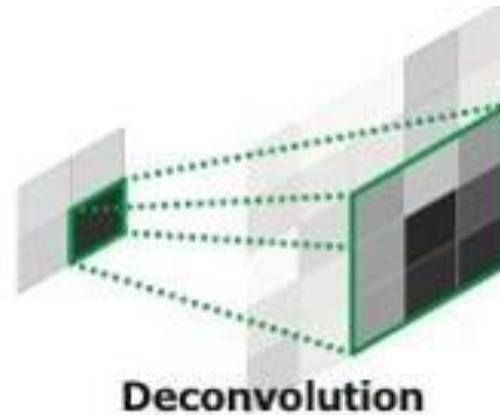
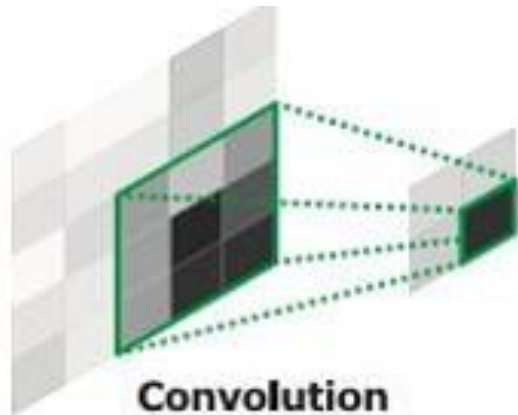
# end-to-end, pixels-to-pixels network

- We need define up-sampling and loss



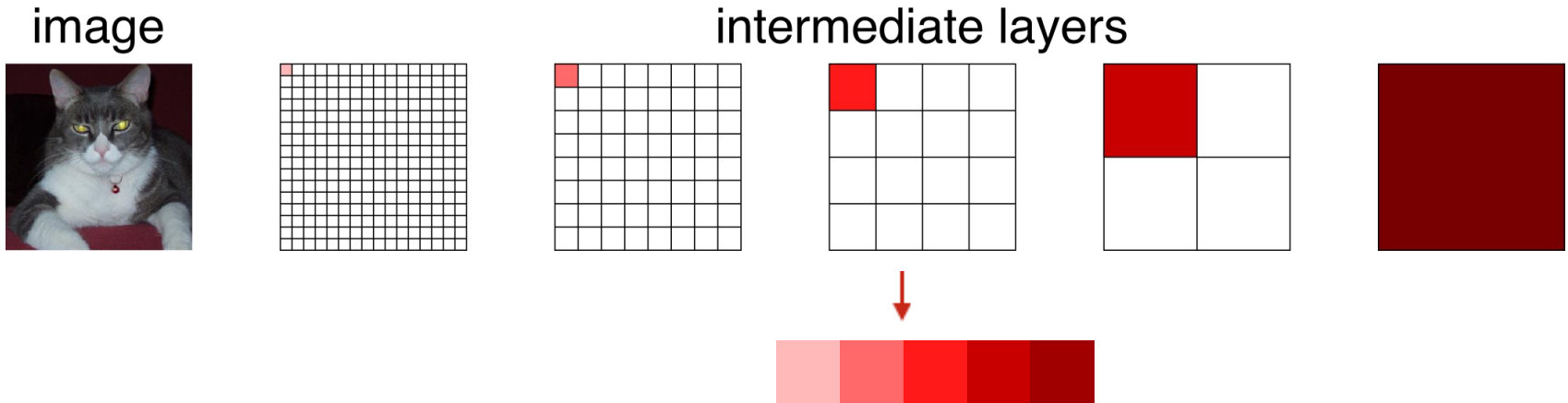
# Upsampling

- Or “deconvolution”
- Bilinear interpolation
- In a sense, upsampling with factor  $f$  is convolution with a *fractional* input stride of  $1/f$ .
- Gradient computation?



# spectrum of deep features

combine *where* (local, shallow) with *what* (global, deep)

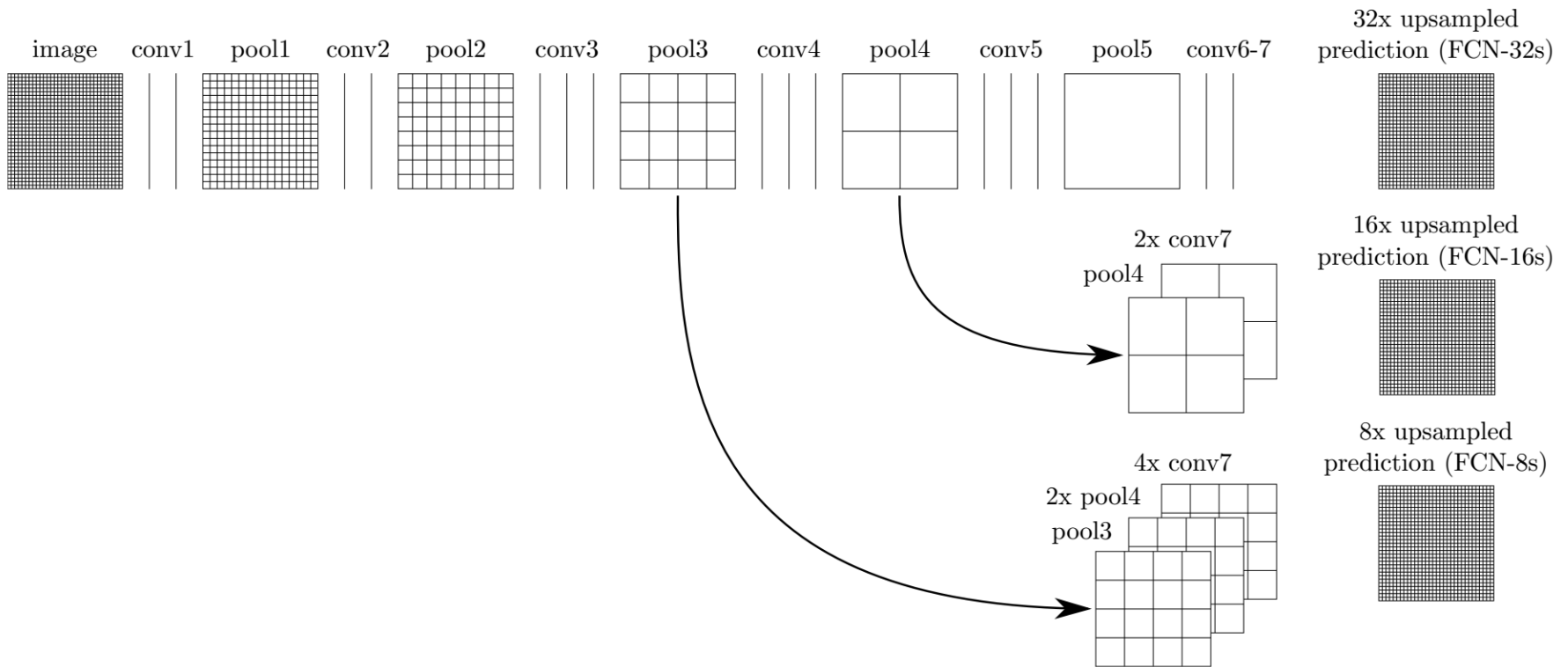


fuse features into deep jet

(cf. Hariharan et al. CVPR15 “hypercolumn” )

# Skip Layer Fusion

- Computing with different strides ("S")



## Other Issues

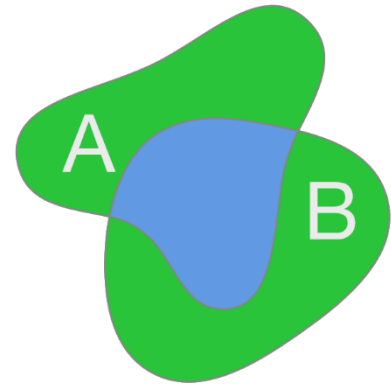
- Per-pixel loss: multinomial logistic loss

$$P(y^{(i)} = k | x^{(i)}; \theta) = \frac{\exp(\theta^{(k)\top} x^{(i)})}{\sum_{j=1}^K \exp(\theta^{(j)\top} x^{(i)})}$$

$$J(\theta) = - \left[ \sum_{i=1}^m \sum_{k=1}^K 1 \{y^{(i)} = k\} \log \frac{\exp(\theta^{(k)\top} x^{(i)})}{\sum_{j=1}^K \exp(\theta^{(j)\top} x^{(i)})} \right]$$

- Evaluation metric: IoU (intersection over union)

$$\frac{\text{true positive}}{(\text{true positive} + \text{false positive} + \text{false negative})}$$



# Comparing FCN-32, FCN-16, FCN-8

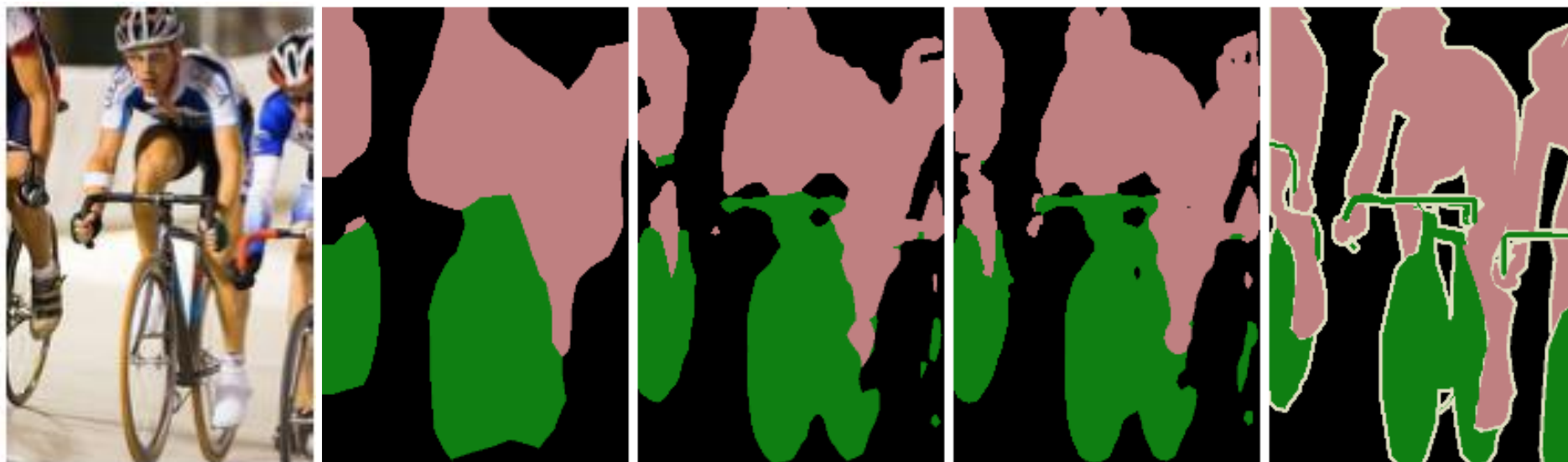
input image

stride 32

stride 16

stride 8

ground truth



no skips

1 skip

2 skips

Table 2. Comparison of skip FCNs on a subset<sup>7</sup> of PASCAL VOC 2011 segval. Learning is end-to-end, except for FCN-32s-fixed, where only the last layer is fine-tuned. Note that FCN-32s is FCN-VGG16, renamed to highlight stride.

|               | pixel<br>acc. | mean<br>acc. | mean<br>IU  | f.w.<br>IU  |
|---------------|---------------|--------------|-------------|-------------|
| FCN-32s-fixed | 83.0          | 59.7         | 45.4        | 72.0        |
| FCN-32s       | 89.1          | 73.3         | 59.4        | 81.4        |
| FCN-16s       | 90.0          | 75.7         | 62.4        | 83.0        |
| FCN-8s        | <b>90.3</b>   | <b>75.9</b>  | <b>62.7</b> | <b>83.2</b> |

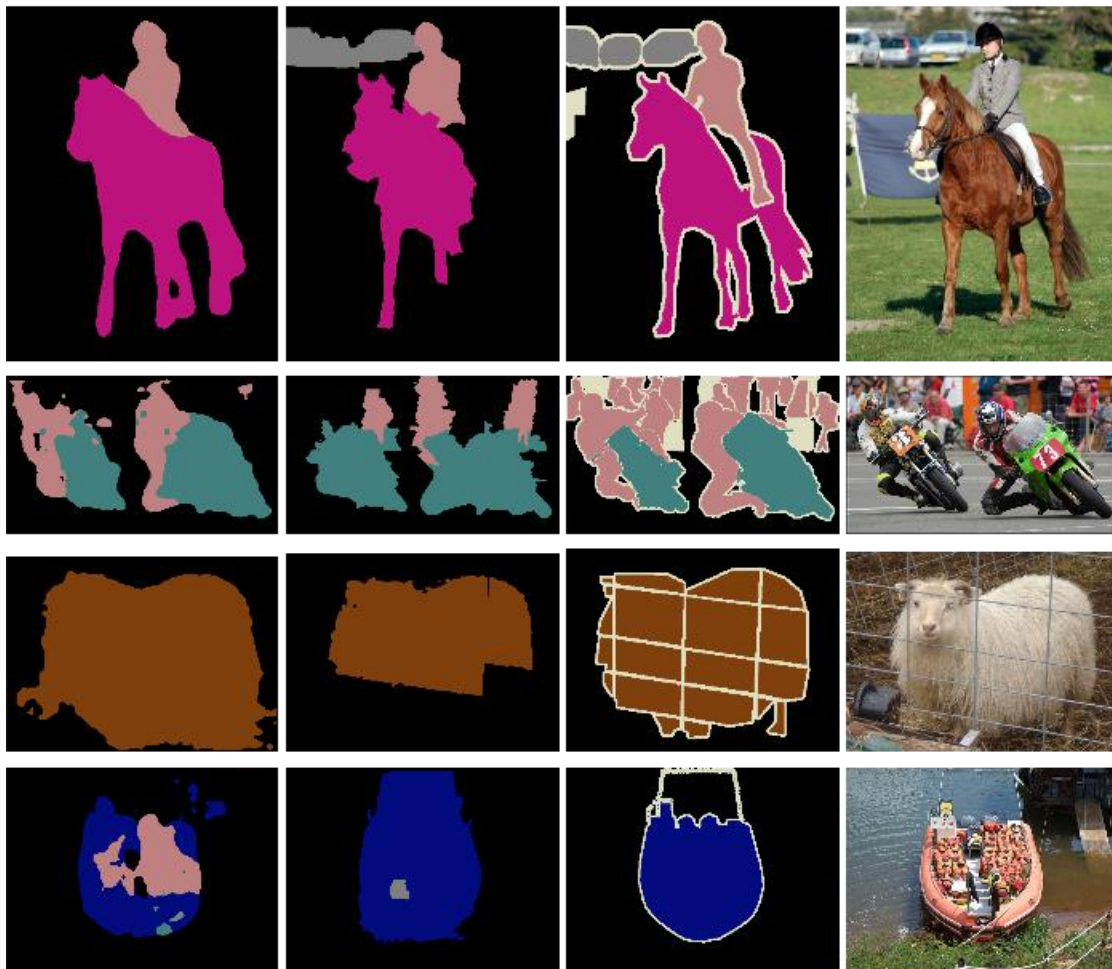
# Experimental Results

FCN

SDS\*

Truth

Input



Relative to prior state-of-the-art SDS:

-30% relative improvement for mean IoU

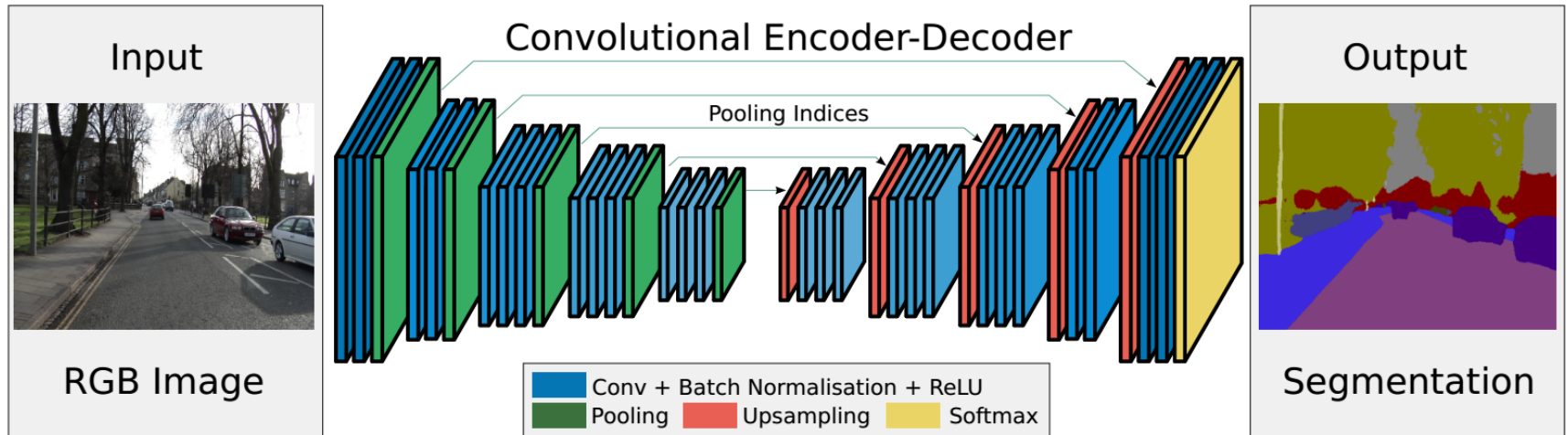
-286 $\times$  faster

\*Simultaneous Detection and Segmentation Hariharan et al. ECCV14



# SegNet

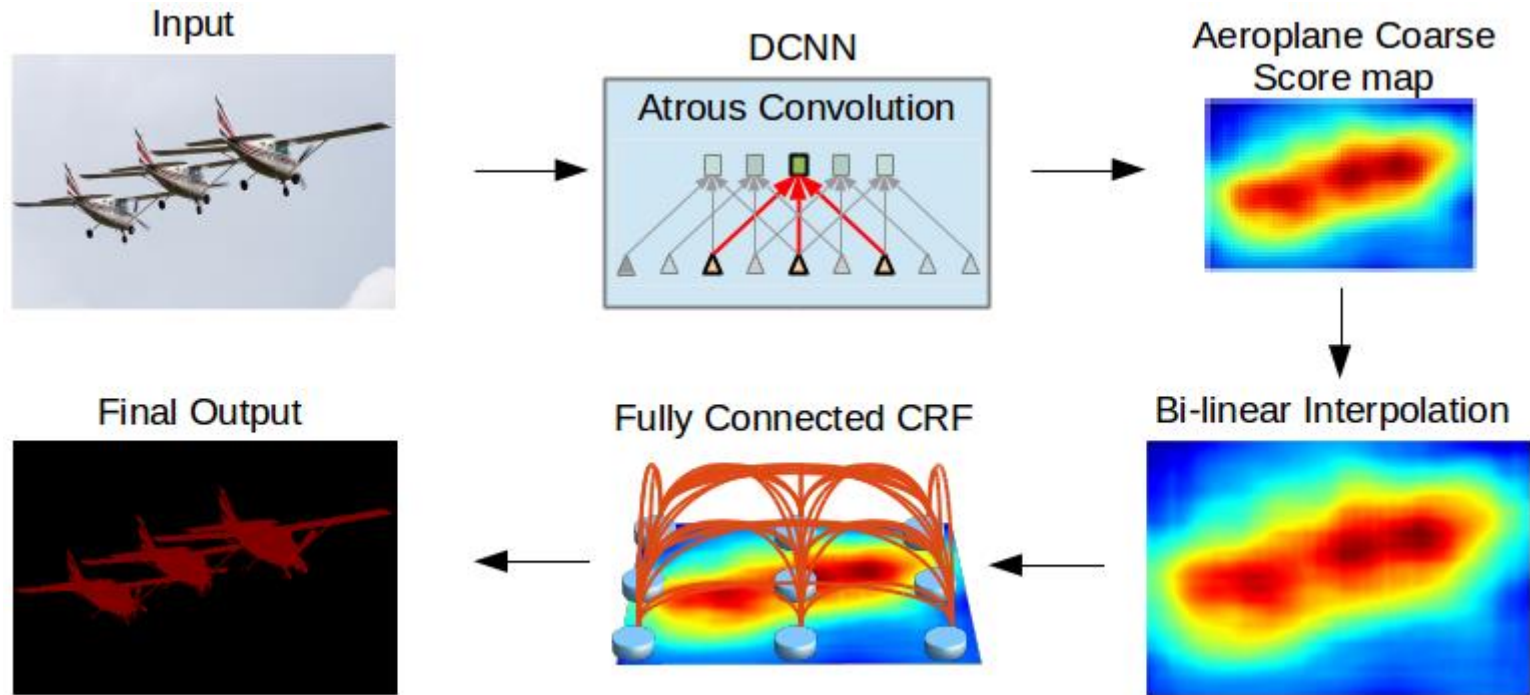
- <http://mi.eng.cam.ac.uk/projects/segnet/>



**Vijay Badrinarayanan, Alex Kendall and Roberto Cipolla "SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation." arXiv preprint arXiv:1511.00561, 2015.**

# DeepLab

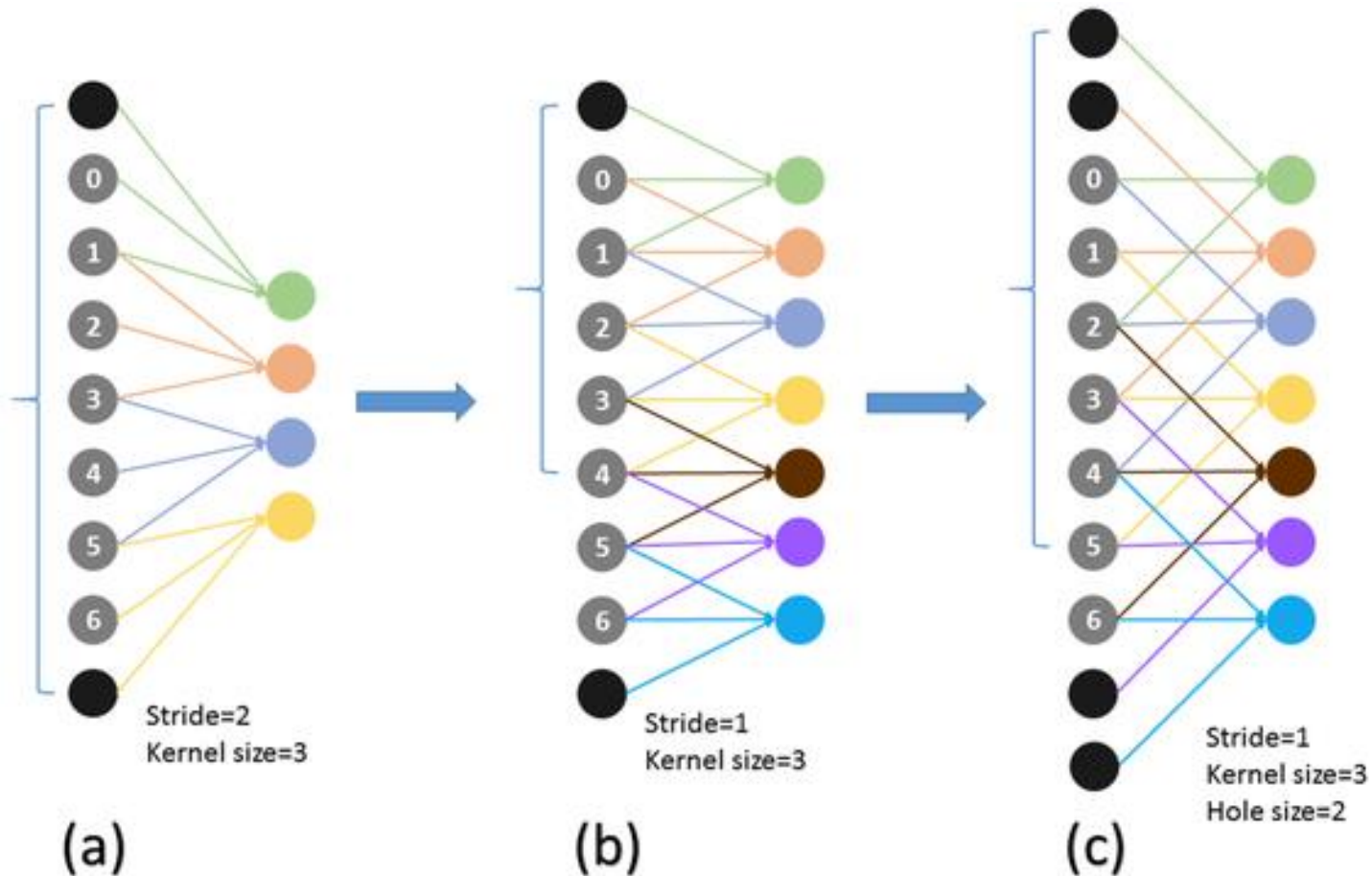
- <http://liangchiehchen.com/projects/DeepLab.html>



Liang-Chieh Chen, George Papandreou et al., Semantic Image Segmentation with Deep Convolutional Nets and Fully Connected CRFs, ICLR 2015

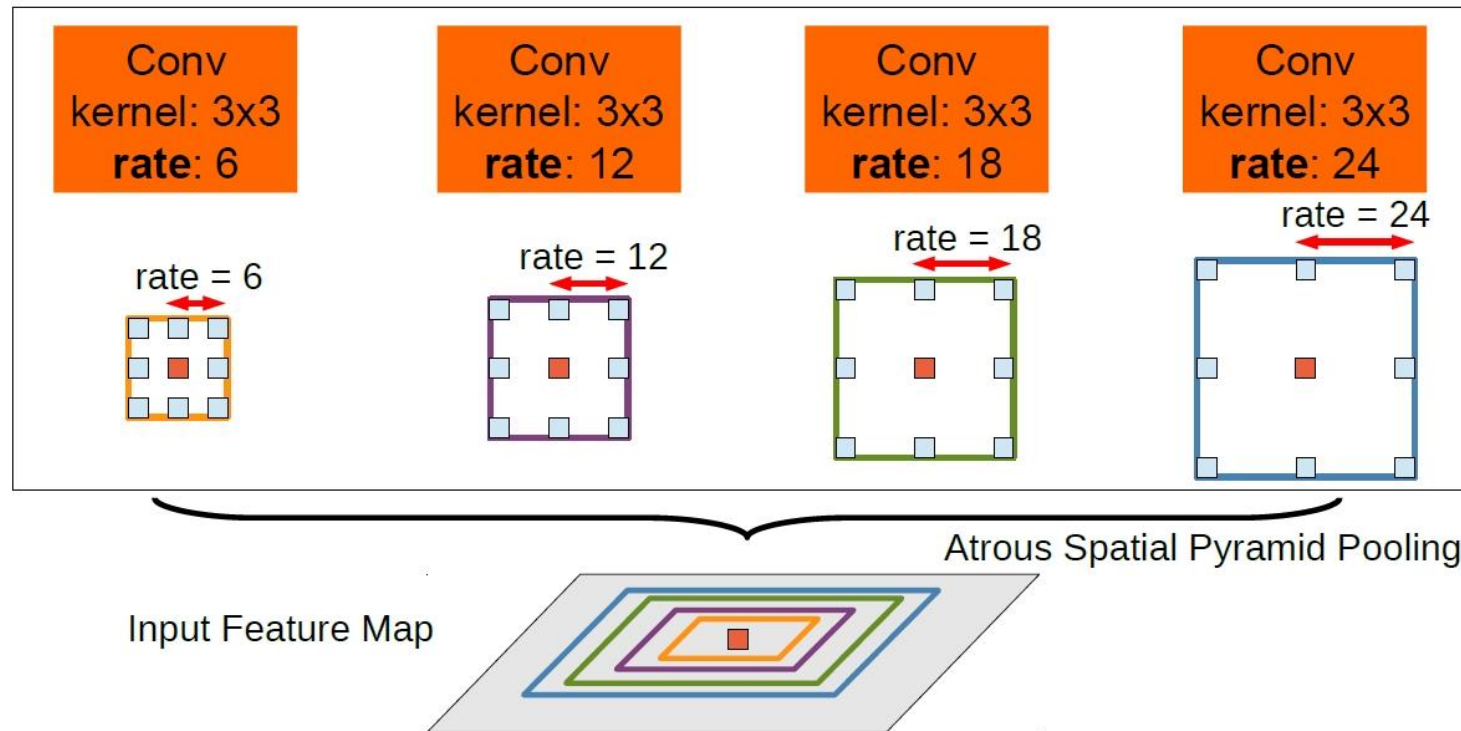
# Atrous Convolution

- Expand the field of view of filters



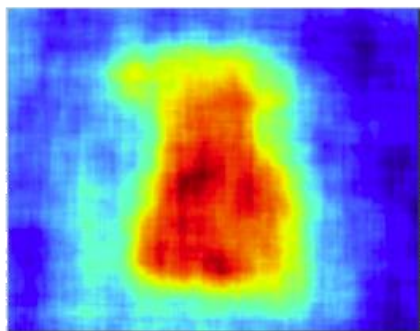
# Atrous Spatial Pyramid Pooling

- To classify the center pixel (orange), Atrous Spatial Pyramid Pooling exploits multi-scale features by employing multiple parallel filters with different rates. The effective Field-Of-VIEWS are shown in different colors.

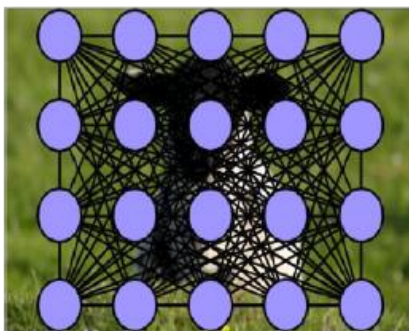
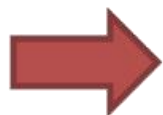


# CRF-as-RNN

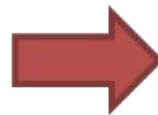
- Conditional Random Fields (CRFs): A CRF can account for contextual information in the image



Coarse output from the pixel-wise classifier



MRF/CRF modelling



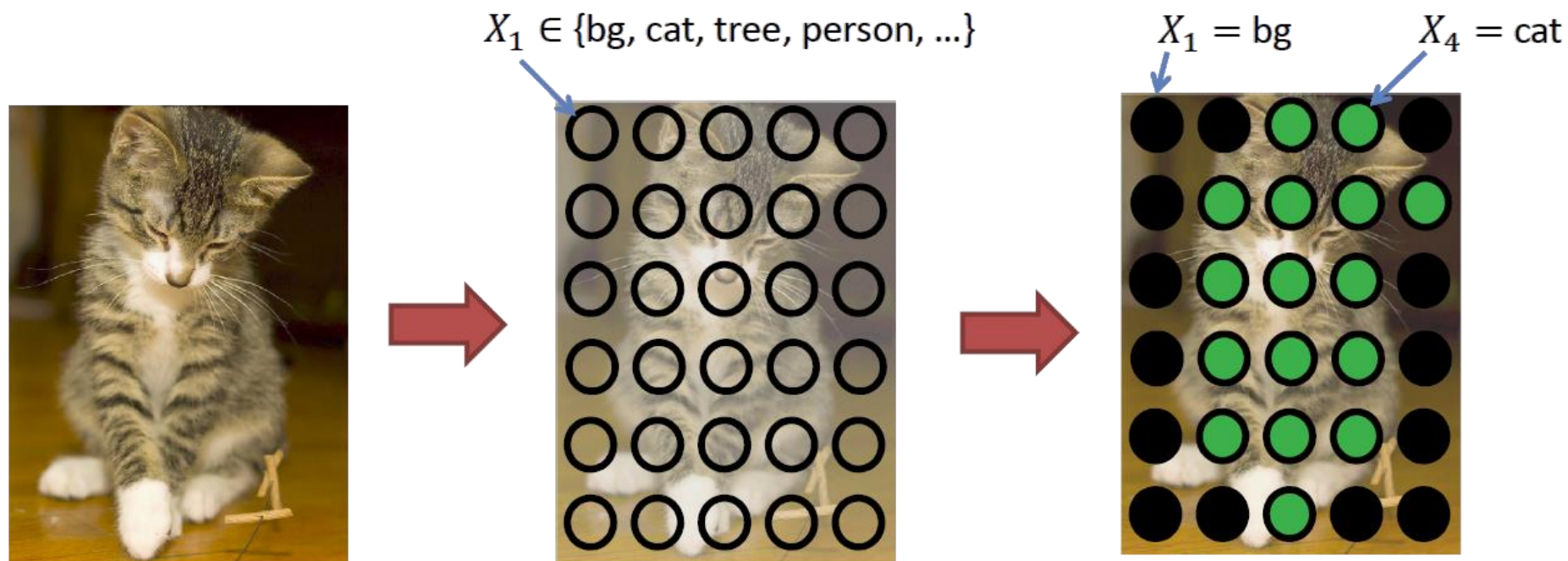
Output after the CRF inference

Shuai Zheng et al., Conditional Random Fields as Recurrent Neural Networks, ICCV 2015

<http://crfasrnn.torr.vision/>



# CNN-as-RNN

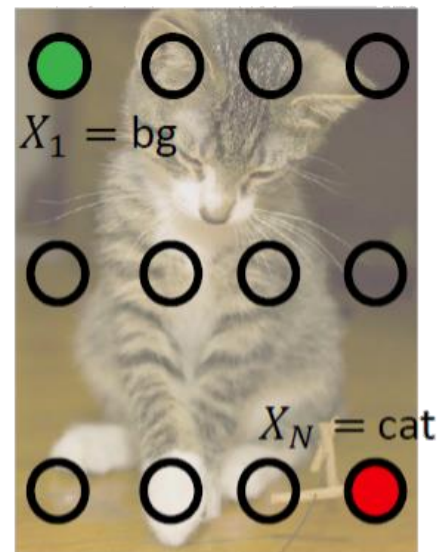


- Define a discrete random variable  $X_i$  for each pixel  $i$ .
- Each  $X_i$  can take a value from the label set.
- Connect random variables to form a random field. (MRF)
- Most probable assignment *given the image*  $\rightarrow$  segmentation.

# CNN-as-RNN

$$\Pr(X_1 = x_1, X_2 = x_2, \dots, X_N = x_N | I) = \Pr(\underline{X} = \underline{x} | I)$$

$$\Pr(\underline{X} = \underline{x} | I) = \exp(-E(\underline{x} | I))$$



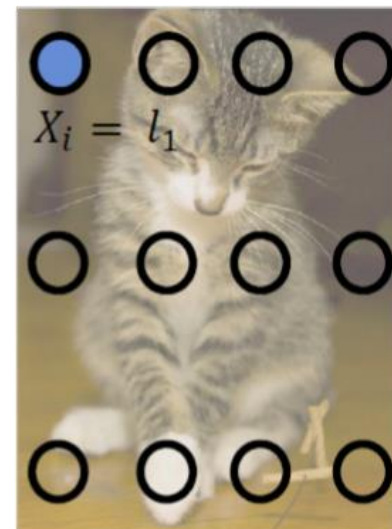
- Maximize  $\Pr(\underline{X} = \underline{x}) \rightarrow$  Minimize  $E(\underline{x})$
- So we have formulated the problem as an energy minimization.

# CNN-as-RNN

$$E(\underline{x}|I) = unary\_cost + pairwise\_cost$$

## Unary energy

- $cost(X_i = l_1) = ?$
- Your label doesn't agree with the initial classifier  $\rightarrow$  you pay a penalty.





# CNN-as-RNN

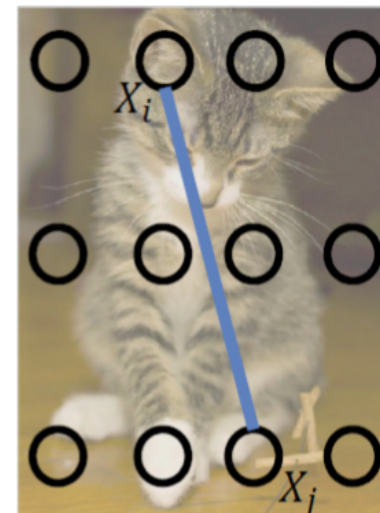
$$E(\underline{x}|I) = unary\_cost + pairwise\_cost$$

## Unary energy

- $cost(X_i = l_1) = ?$
- Your label doesn't agree with the initial classifier  $\rightarrow$  you pay a penalty.

## Pairwise energy

- $cost(X_i = l_1, X_j = l_2) = ?$
- You assign different labels to two very *similar* pixels  $\rightarrow$  you pay a penalty.
- How do you measure *similarity*?



# CNN-as-RNN

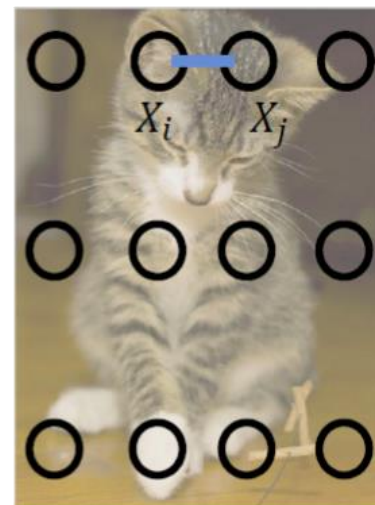
$$E(\underline{x}|I) = unary\_cost + pairwise\_cost$$

## Unary energy

- $cost(X_i = l_1) = ?$
- Your label doesn't agree with the initial classifier  $\rightarrow$  you pay a penalty.

## Pairwise energy

- $cost(X_i = l_1, X_j = l_2) = ?$
- You assign different labels to two very *similar* pixels  $\rightarrow$  you pay a penalty.
- How do you measure similarity?



# CRF-as-RNN

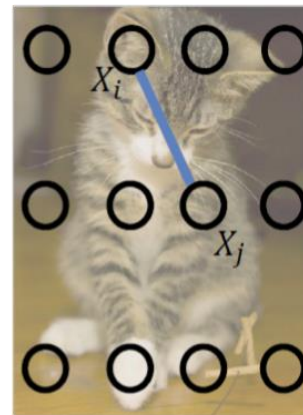
$$E(\underline{x}|I) = unary\_cost + pairwise\_cost$$

## Unary energy

- $cost(X_i = l_1) = ?$
- Your label doesn't agree with the initial classifier  $\rightarrow$  you pay a penalty.

## Pairwise energy

- $cost(X_i = l_1, X_j = l_2) = ?$
- You assign different labels to two very *similar* pixels  $\rightarrow$  you pay a penalty.
- How do you measure similarity?



## Dense CRF Formulation

[Krähenbühl & Koltun, NIPS 2011.]

- Pairwise energies are defined for every pixel pair in the image.

$$E(\underline{x}) = \sum_i unary(x_i) + \sum_{i,j} pairwise(x_i, x_j)$$

- Exact inference is not feasible.
- Use approximate mean field inference.

$$\exp(-E(\underline{X})) = P(\underline{X}) = \prod_{i=1}^N Q(X_i)$$

# Fully Connected CRFs as a CNN

$Q_i \leftarrow \frac{1}{Z_i} \exp(U_i(l))$  for all  $i$  ▷ Initialization

**while** not converged **do**

$\tilde{Q}_i^{(m)}(l) \leftarrow \sum_{j \neq i} k^{(m)}(\mathbf{f}_i, \mathbf{f}_j) Q_j(l)$  for all  $m$

▷ Message Passing

$\check{Q}_i(l) \leftarrow \sum_m w^{(m)} \tilde{Q}_i^{(m)}(l)$

▷ Weighting Filter Outputs

$\hat{Q}_i(l) \leftarrow \sum_{l' \in \mathcal{L}} \mu(l, l') \check{Q}_i(l')$

▷ Compatibility Transform

$\check{\check{Q}}_i(l) \leftarrow U_i(l) - \hat{Q}_i(l)$

▷ Adding Unary Potentials

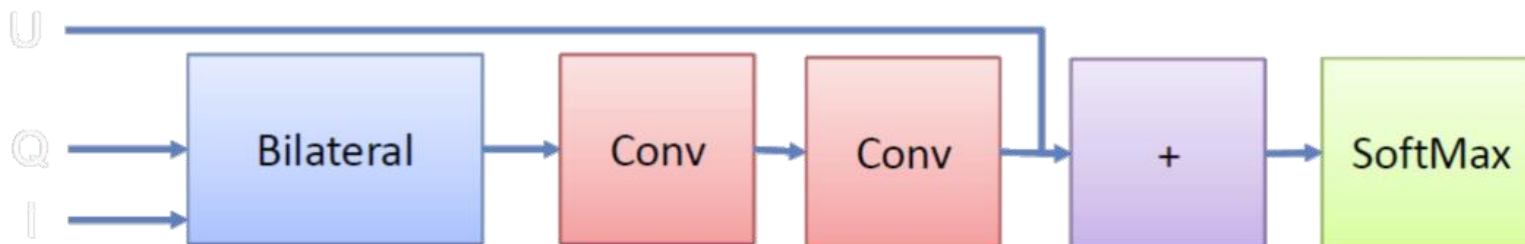
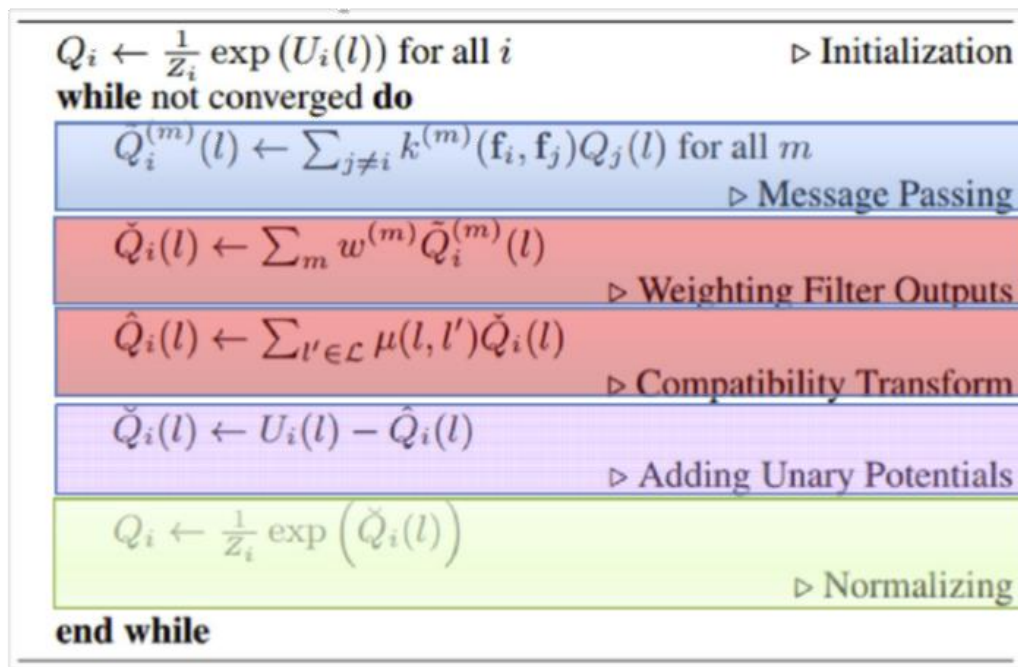
$Q_i \leftarrow \frac{1}{Z_i} \exp(\check{\check{Q}}_i(l))$

▷ Normalizing

**end while**

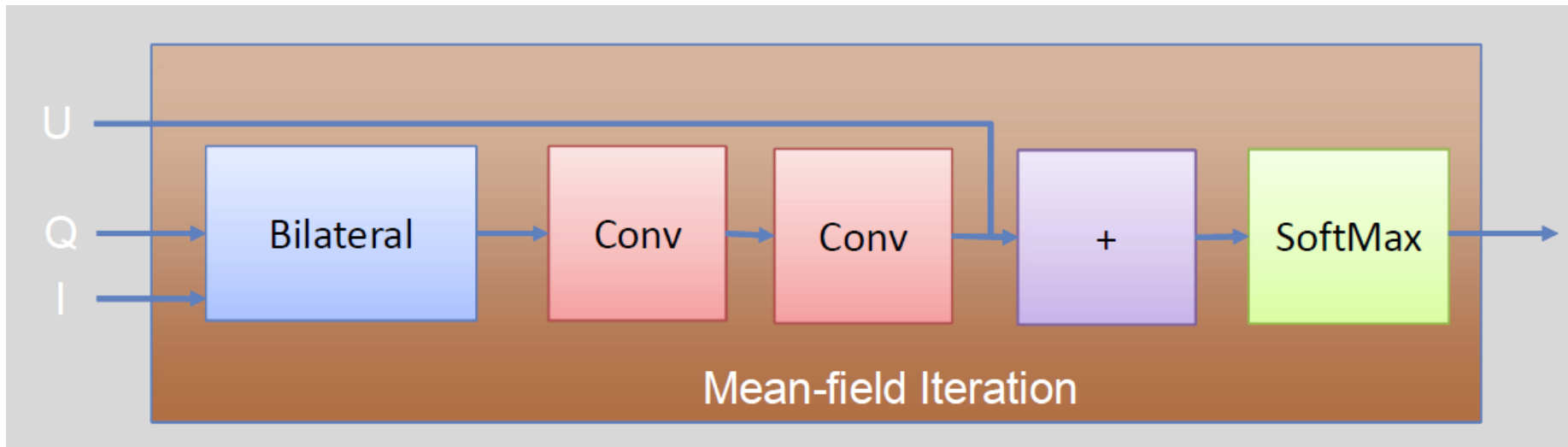
---

# Fully Connected CRFs as a CNN



# CRF as a Recurrent Neural Network

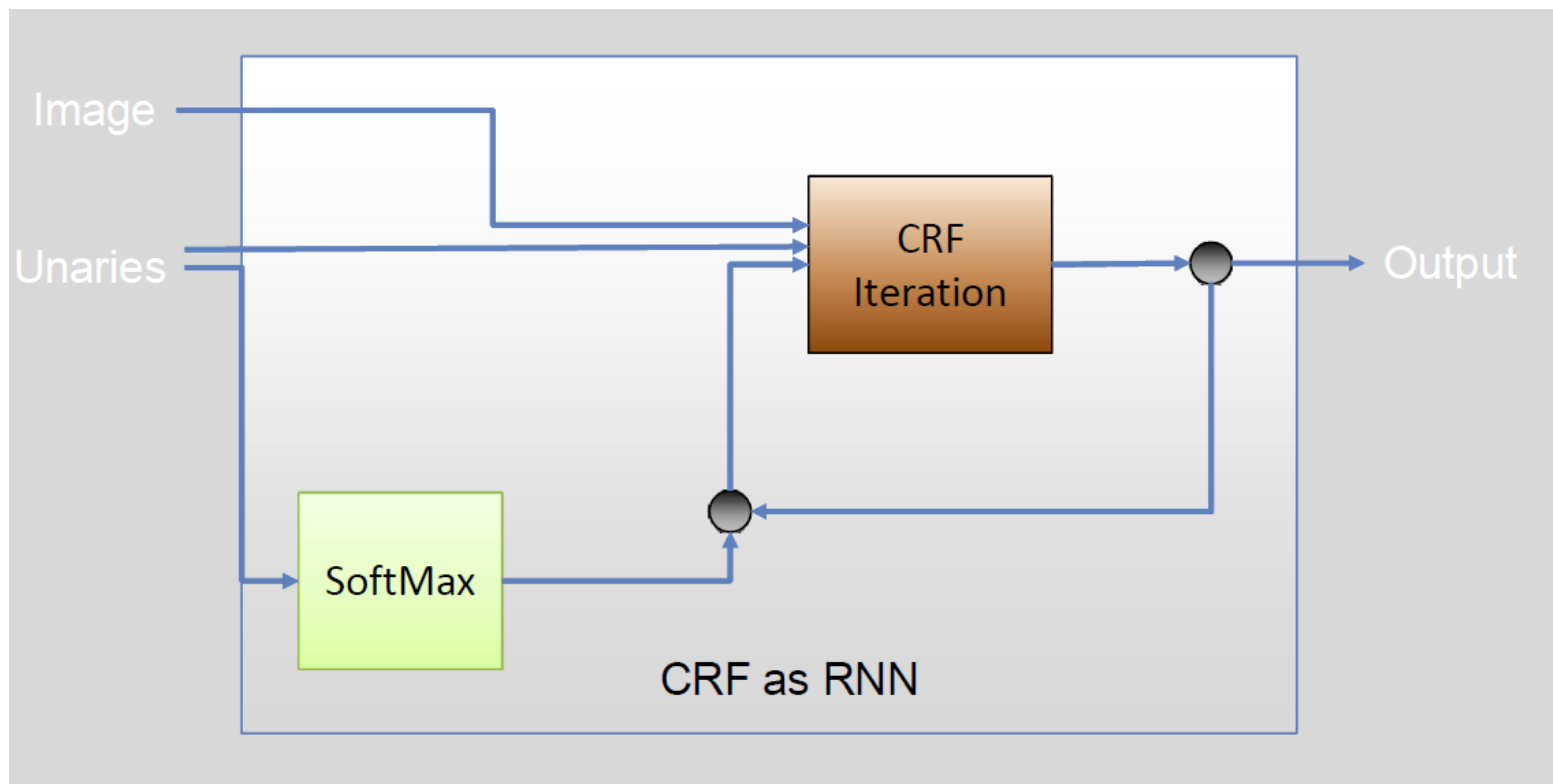
- Each of these blocks is differentiable -> we can propagate



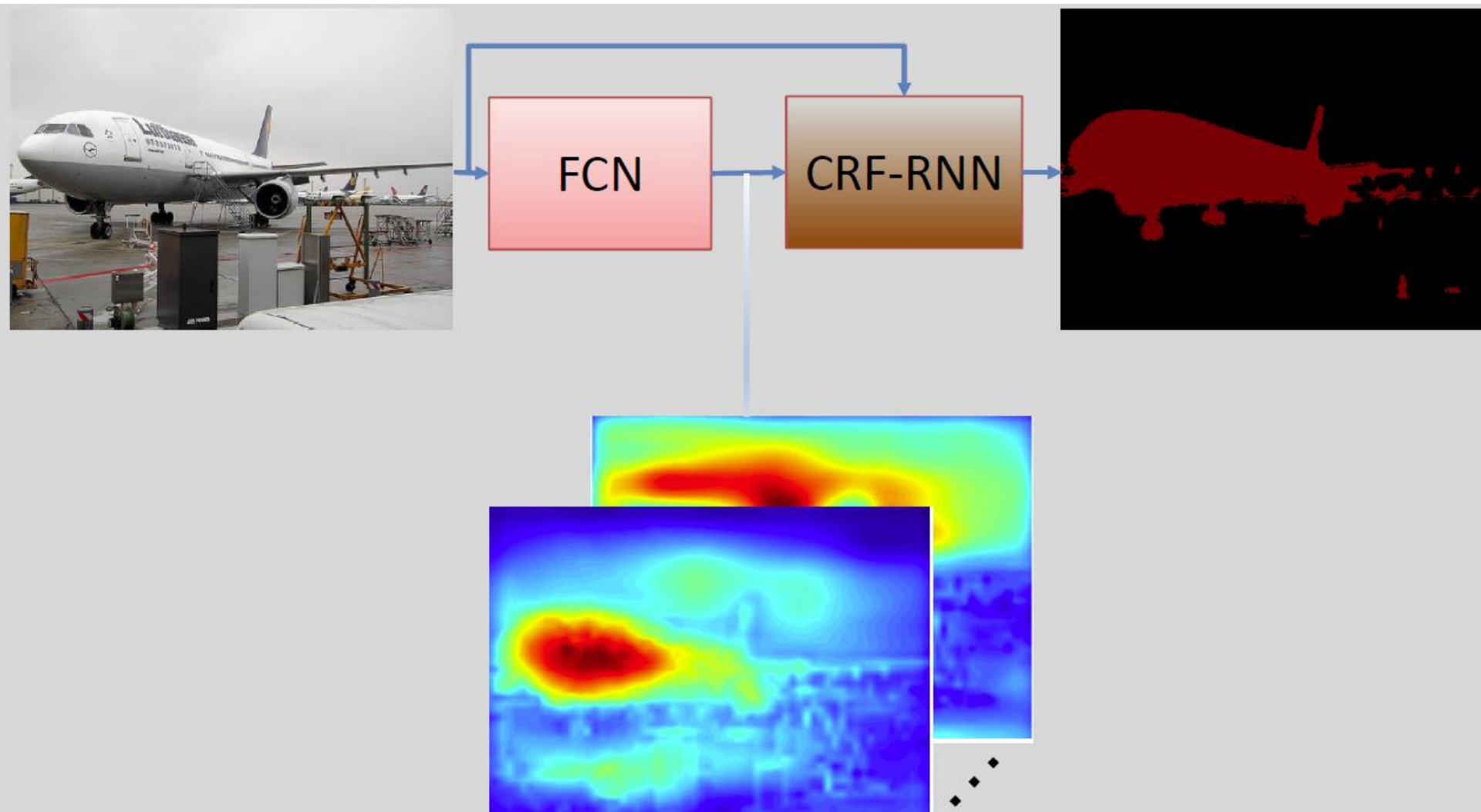


# CRF as a Recurrent Neural Network

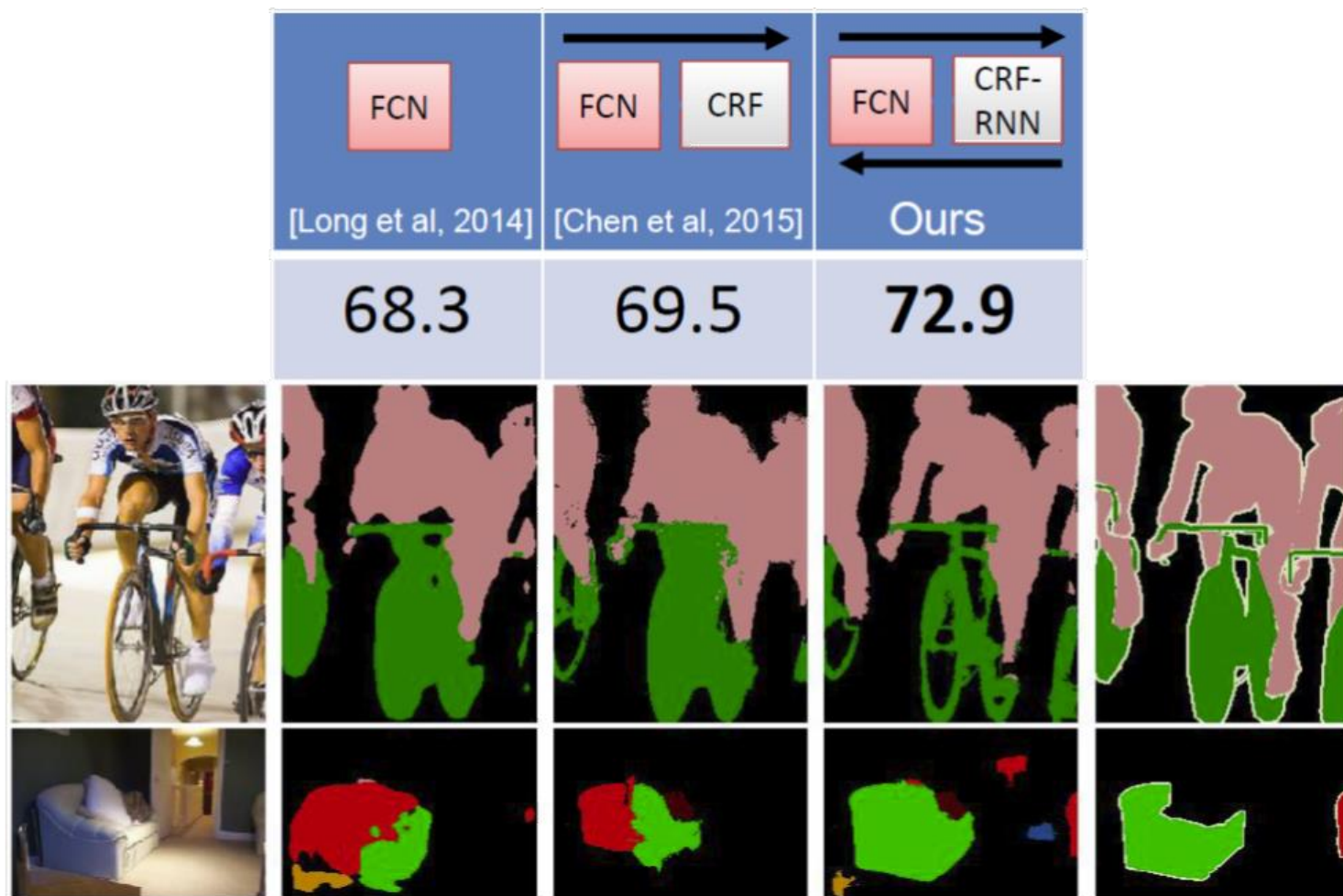
- Each of these blocks is differentiable -> we can propagate



# Putting Things Together



# Experiments



# More Results



Original image (hover to highlight segmented parts)



Semantic segmentation

Objects appearing in the image:

Bicycle

Person



Original image (hover to highlight segmented parts)



Semantic segmentation

Objects appearing in the image:

Bicycle

Bottle

Chair

Dining table

Person

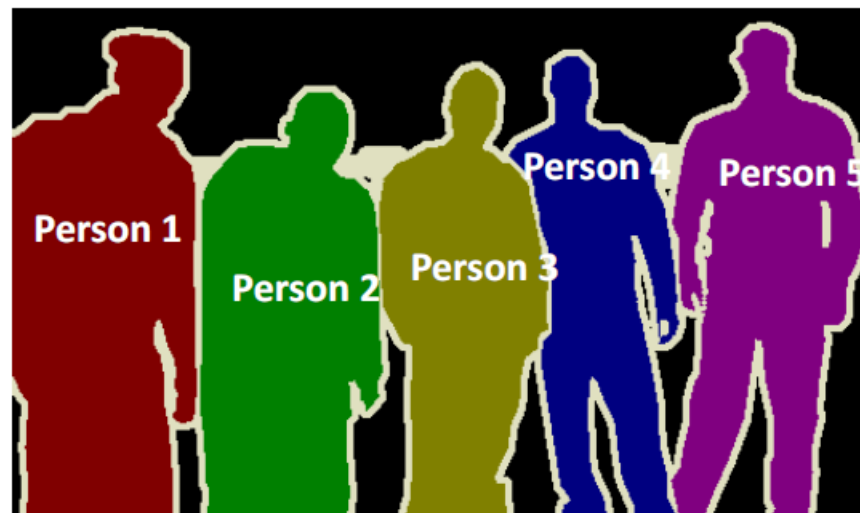


# Mask R-CNN

- Instance-level segmentation



Object Detection



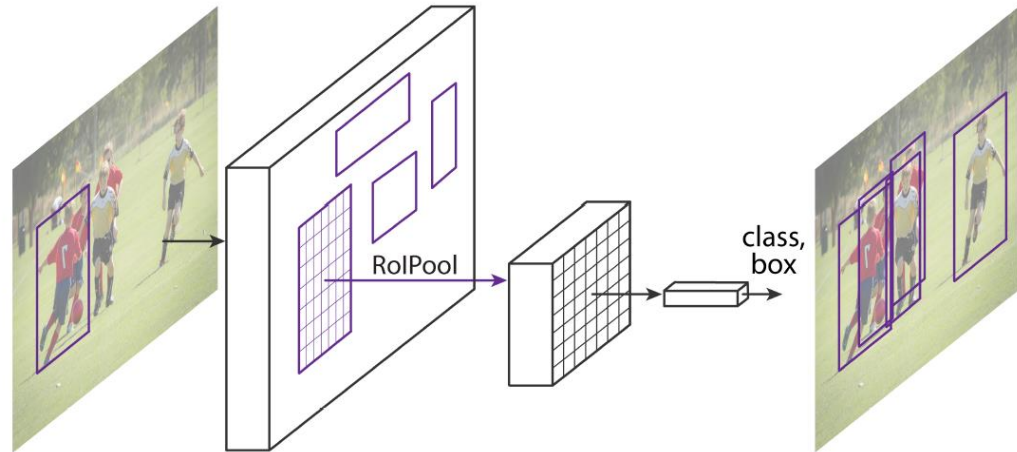
Instance Segmentation



# Retrospect of Detection / Segmentation Models

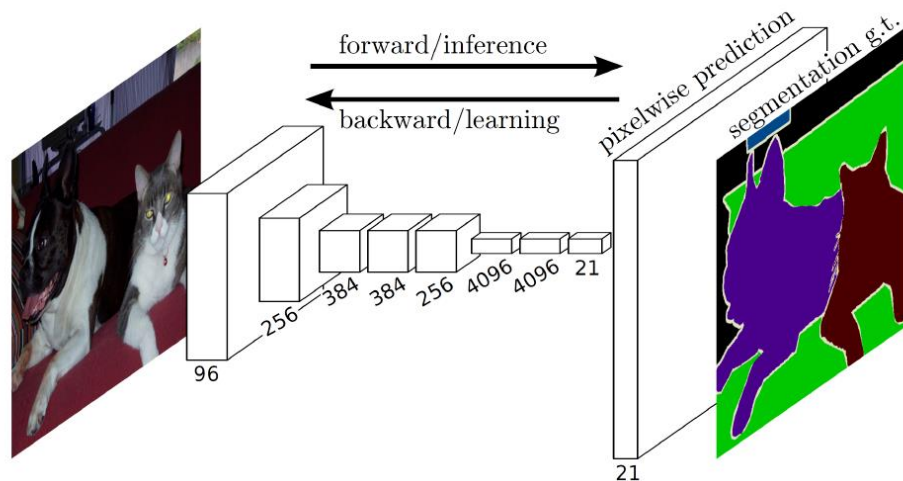
- Fast/Faster R-CNN

- ✓ Good speed
- ✓ Good accuracy
- ✓ Intuitive
- ✓ Easy to use



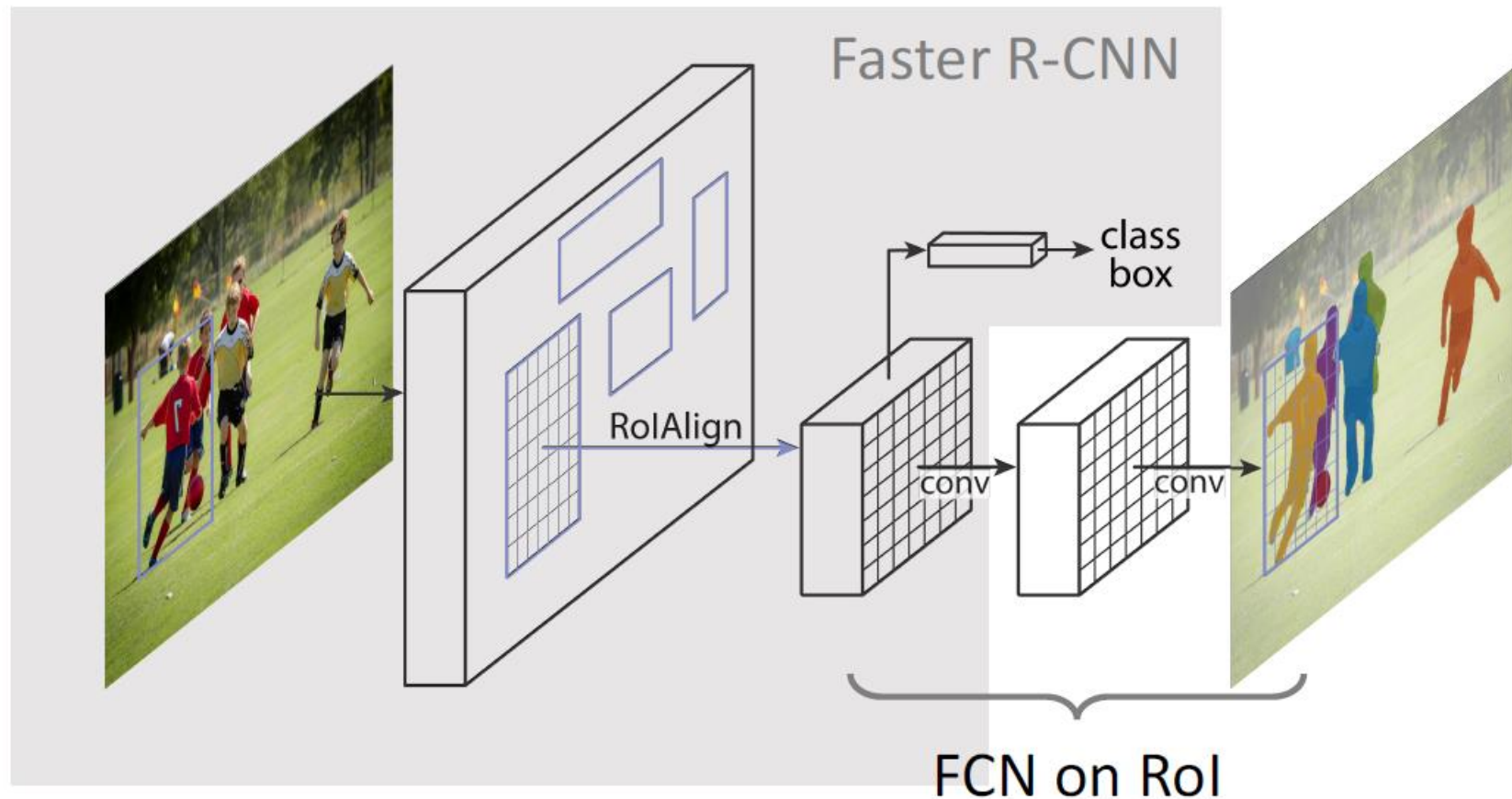
- Fully Convolutional Net (FCN)

- ✓ Good speed
- ✓ Good accuracy
- ✓ Intuitive
- ✓ Easy to use



# Mask R-CNN

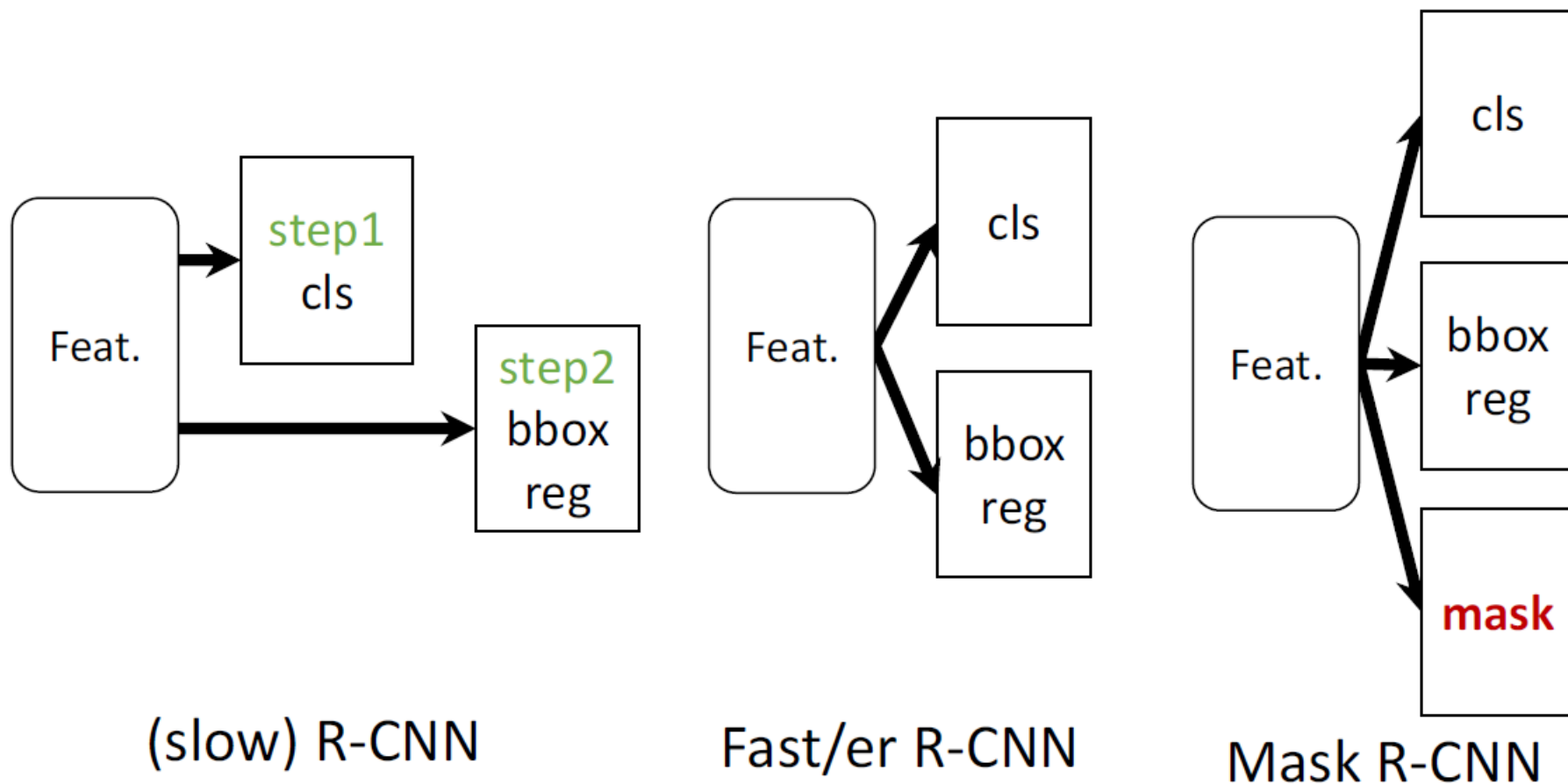
- Mask R-CNN = **Faster R-CNN** with **FCN** on Rols





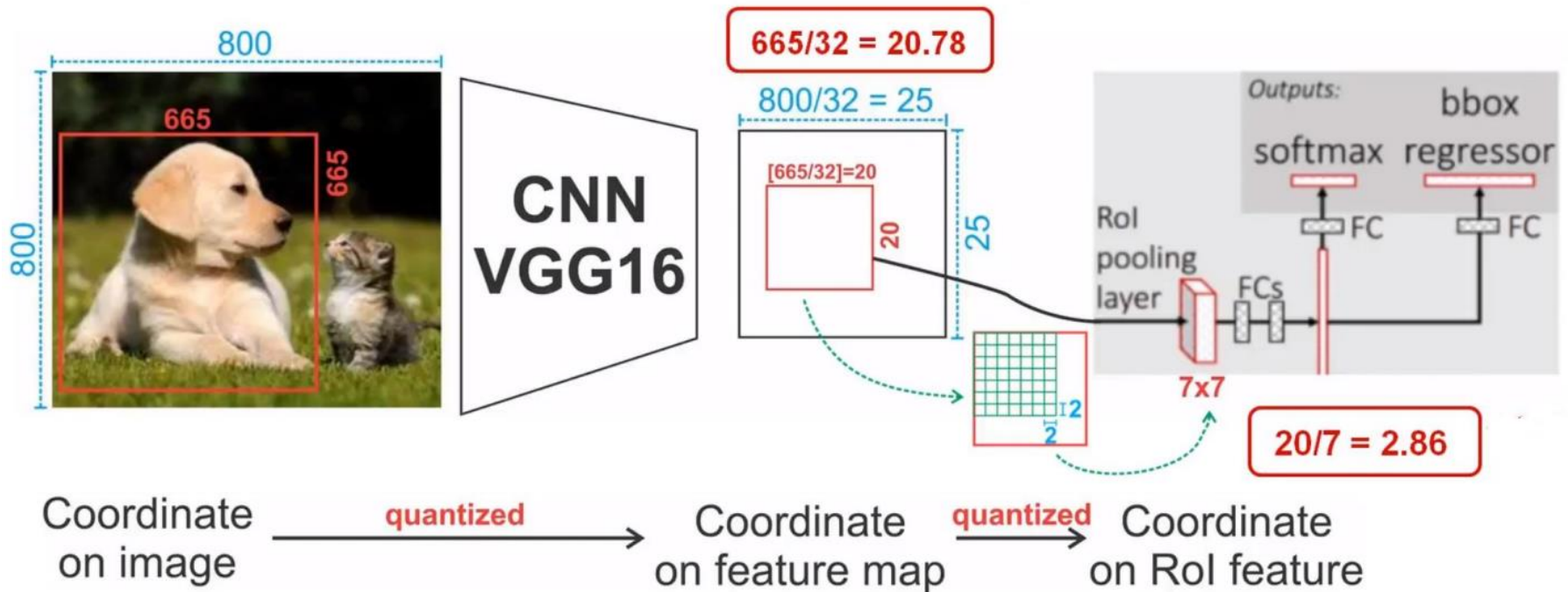
# Mask R-CNN

- Parallel Heads



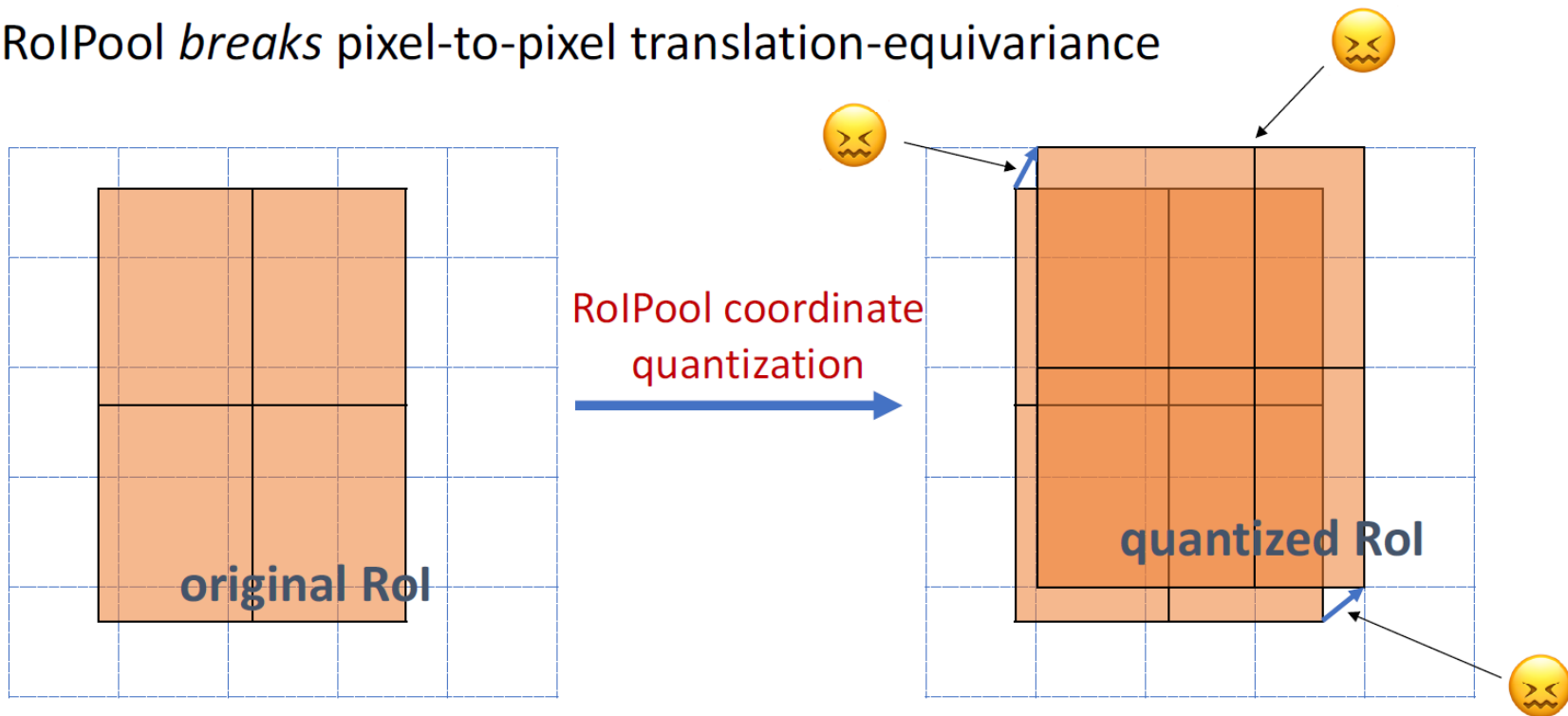
# RoIAlign vs. RoIPool

- Motivation of ROIAlign



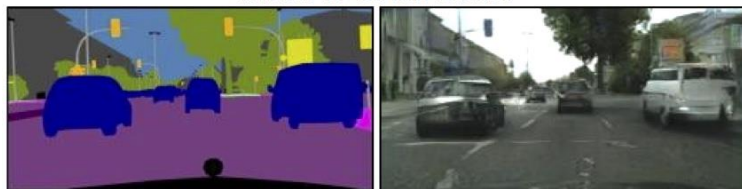
# RoIAlign vs. RoIPool

- RoIPool *breaks* pixel-to-pixel translation-equivariance



# Pix2Pix

Labels to Street Scene



input

output

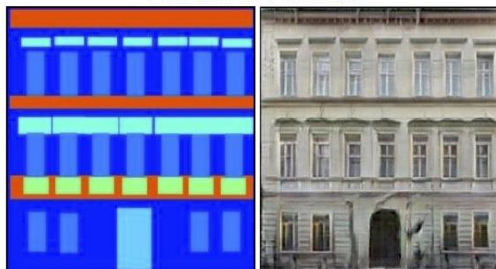
Aerial to Map



input

output

Labels to Facade



input

output

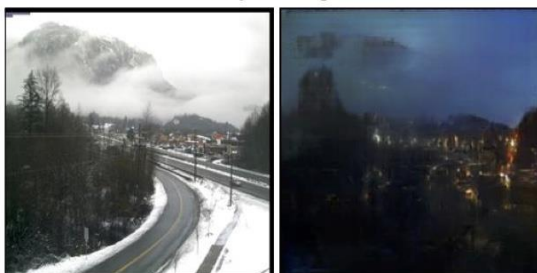
BW to Color



input

output

Day to Night



input

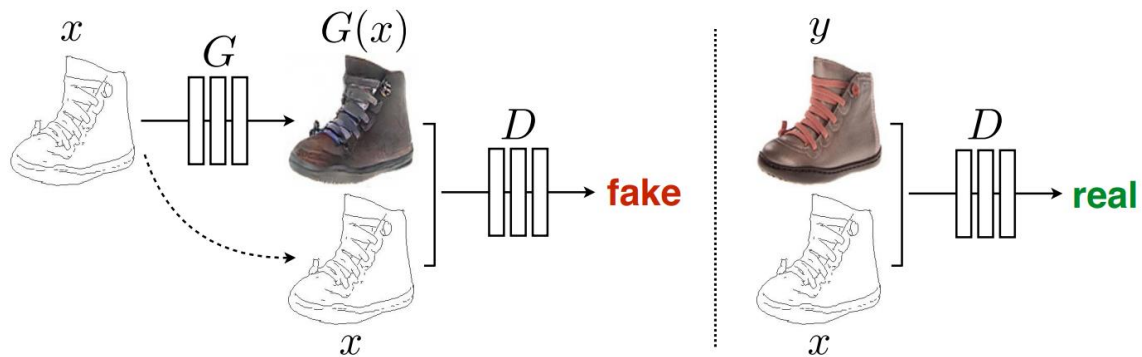
output

Edges to Photo



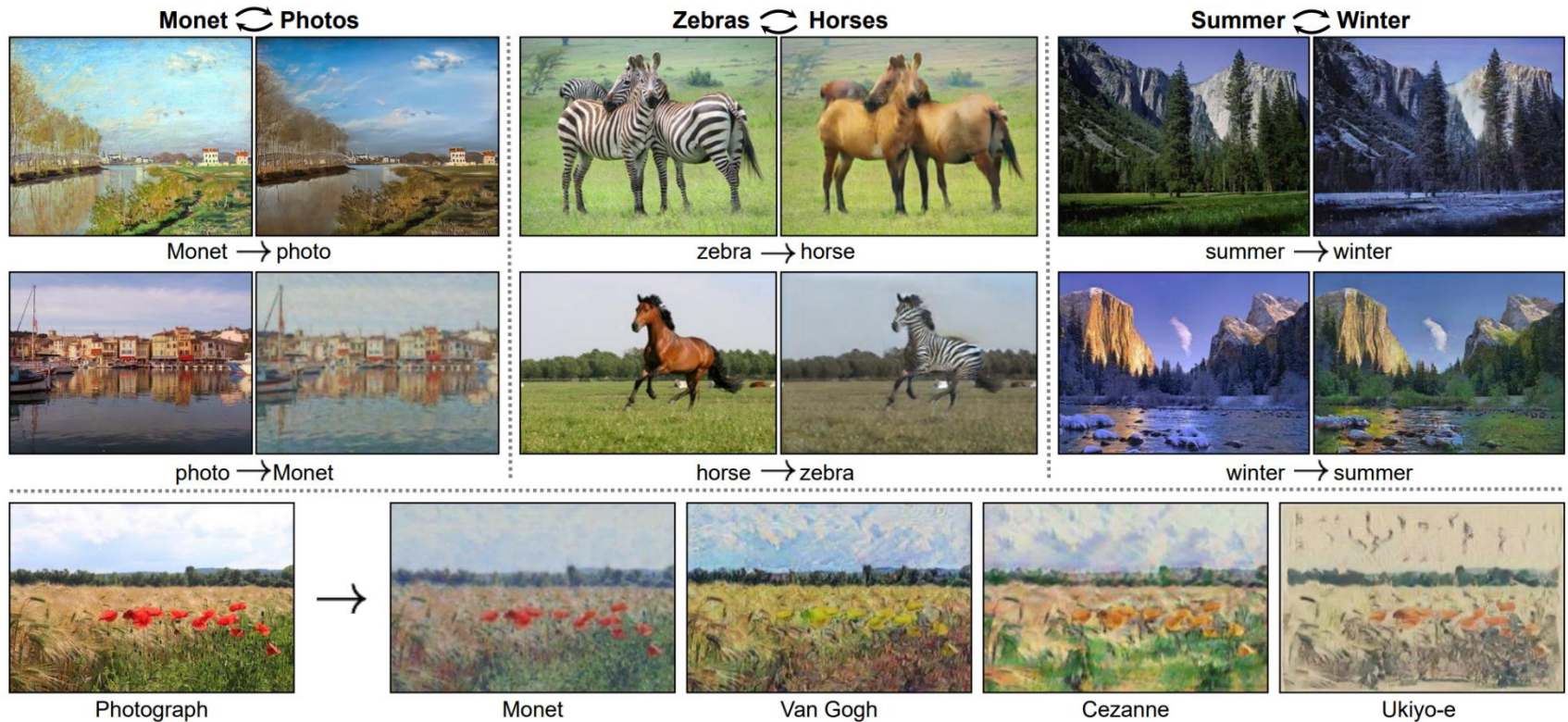
input

output



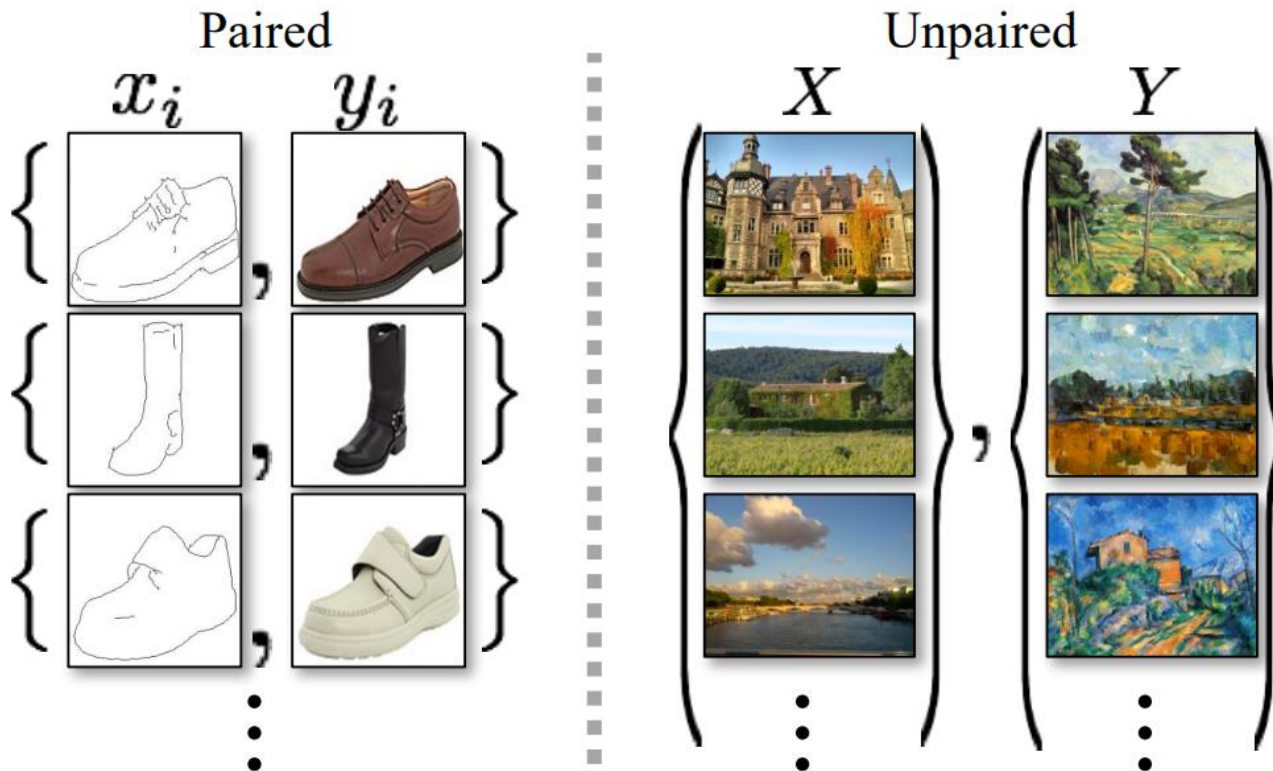


# Cycle-Consistent Adversarial Networks



Jun-Yan Zhu\*, Taesung Park\*, Phillip Isola, Alexei A. Efros. Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. arXiv, 2017.

# Cycle-Consistent Adversarial Networks



Jun-Yan Zhu\*, Taesung Park\*, Phillip Isola, Alexei A. Efros. Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. arXiv, 2017.



# Cycle-Consistent Adversarial Networks

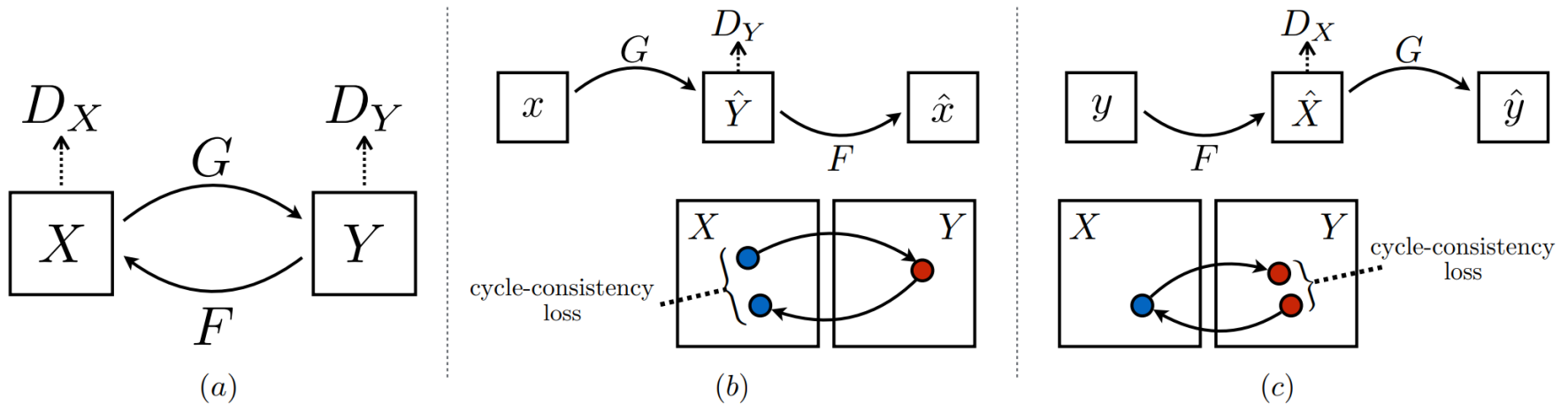
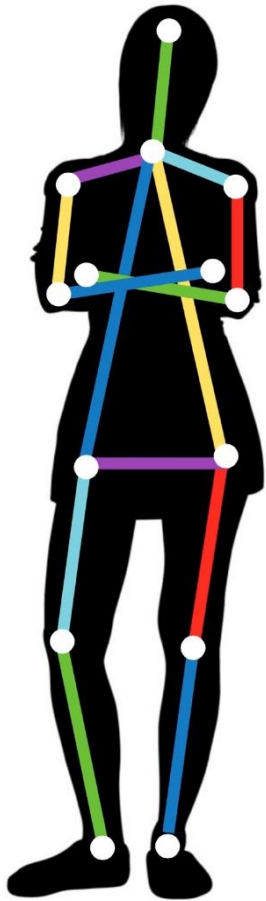


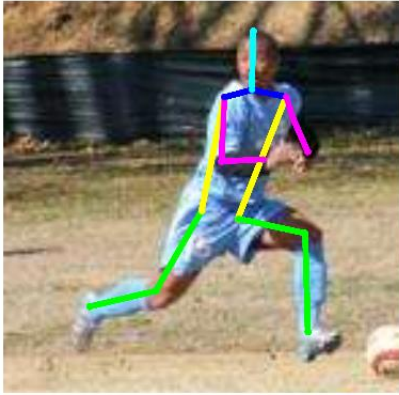
Figure 3: (a) Our model contains two mapping functions  $G : X \rightarrow Y$  and  $F : Y \rightarrow X$ , and associated adversarial discriminators  $D_Y$  and  $D_X$ .  $D_Y$  encourages  $G$  to translate  $X$  into outputs indistinguishable from domain  $Y$ , and vice versa for  $D_X$  and  $F$ . To further regularize the mappings, we introduce two *cycle consistency losses* that capture the intuition that if we translate from one domain to the other and back again we should arrive at where we started: (b) forward cycle-consistency loss:  $x \rightarrow G(x) \rightarrow F(G(x)) \approx x$ , and (c) backward cycle-consistency loss:  $y \rightarrow F(y) \rightarrow G(F(y)) \approx y$

# Human Pose Estimation

- Estimate pre-defined human joints from a single image



# Human Pose Estimation



Action recognition



- null
- shorts
- shoes
- purse
- top
- necklace
- hair
- skin

Clothing Parsing



Human tracking



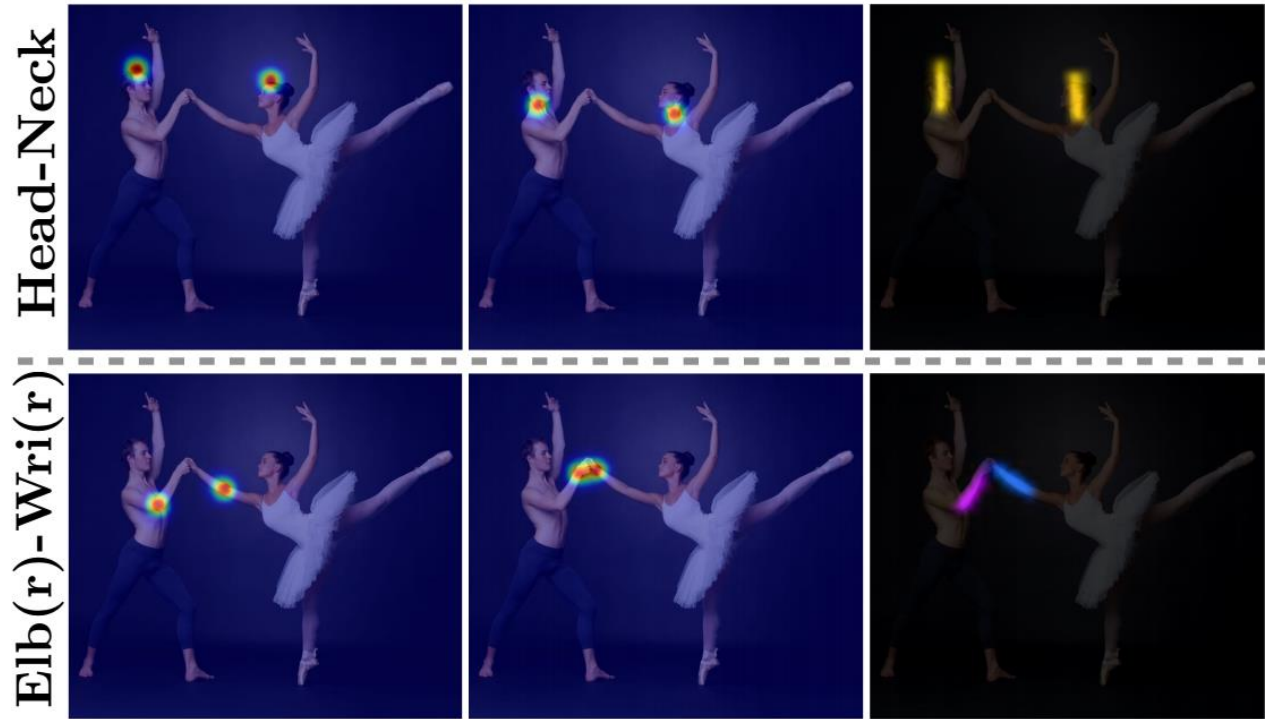
Gaming

# Open Pose

- A Real-time approach for multi-person 2D pose estimation
- Two-branch deep neural networks
- Re-calibrate the state-of-the-art on MPII and COCO-keypoint



(a) Input image



(b) Confidence maps

(c) PAFs

# Open Pose

- Part Affinity Field



(b) Body Parts Confidence Maps

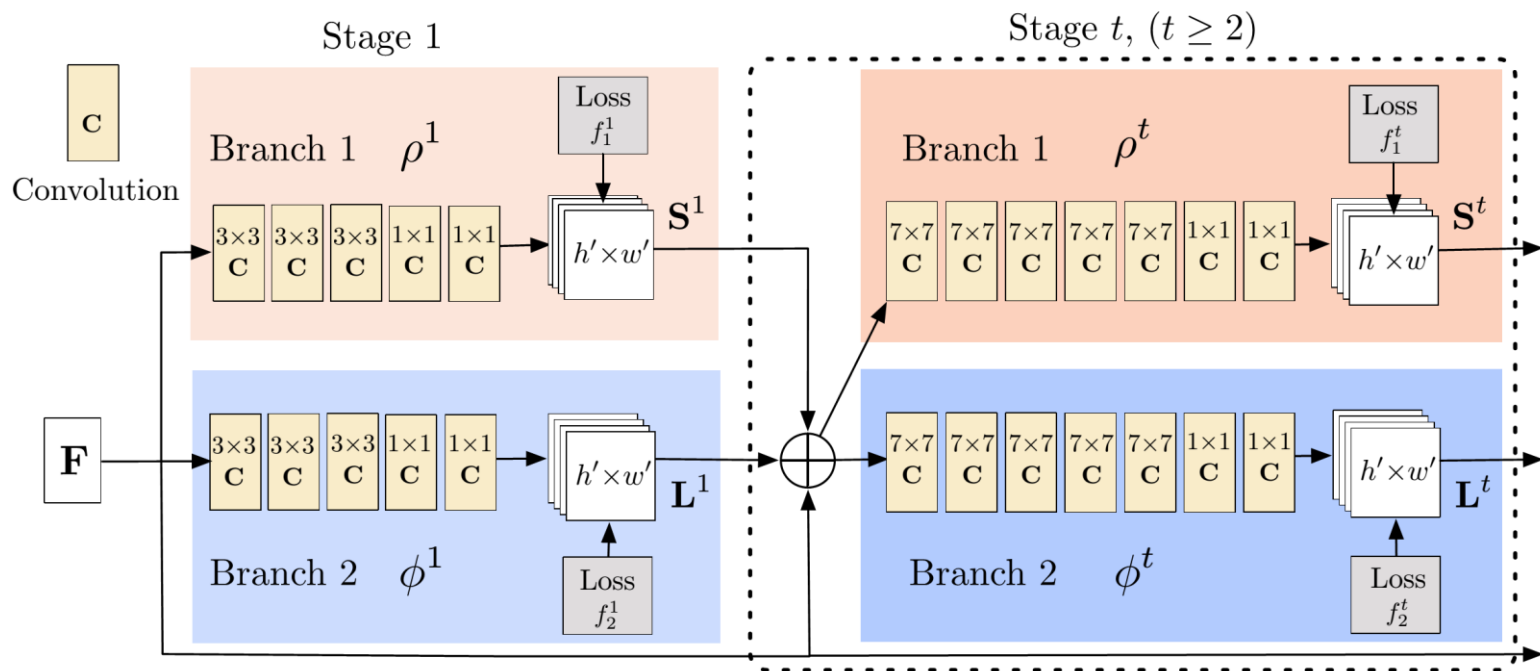


(d) Parts Affinity Fields (PAFs)



# Open Pose

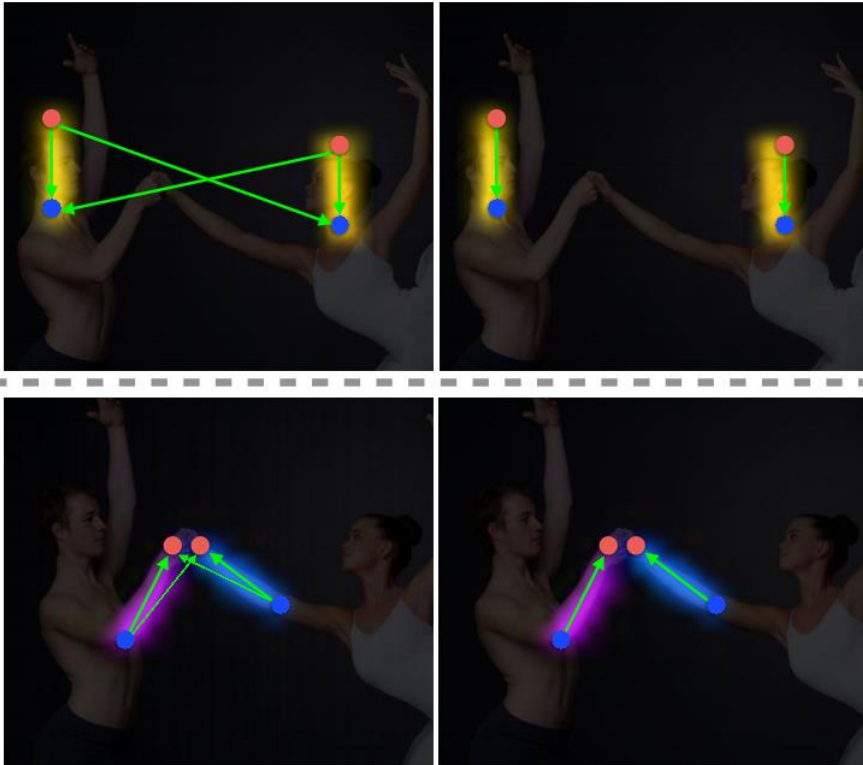
- Network design





# Open Pose

- Bipartite Matching



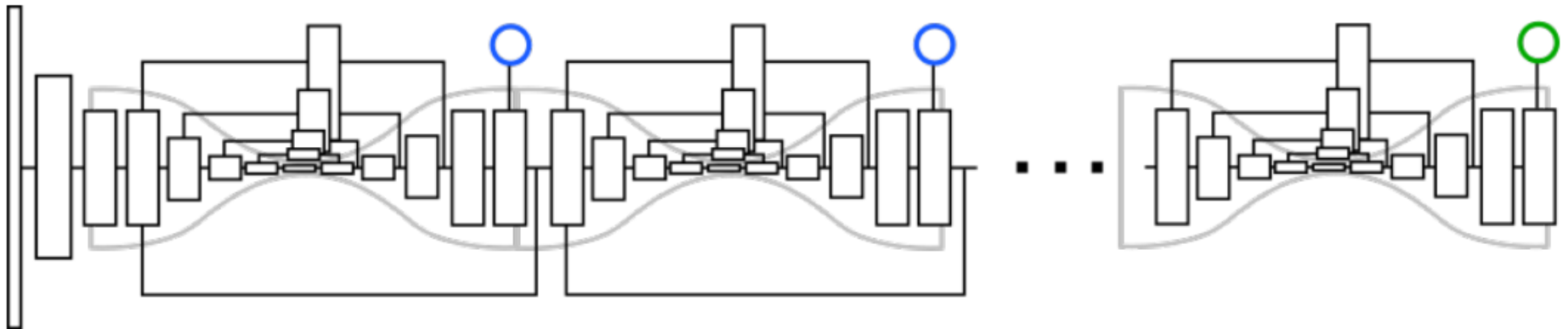
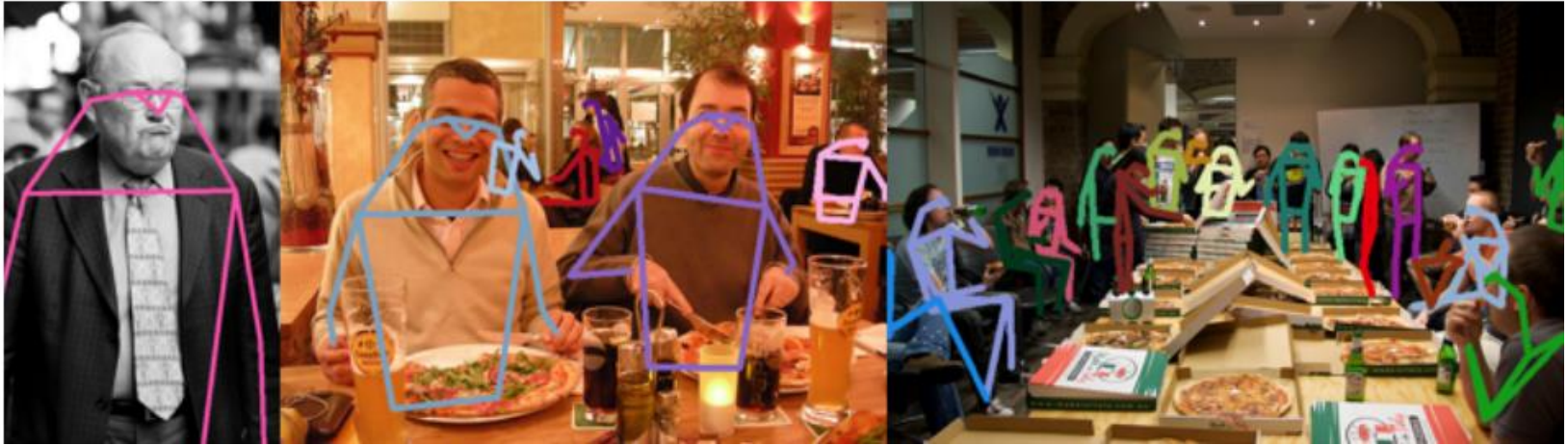
(d) Bipartite matching



(e) Parsing results

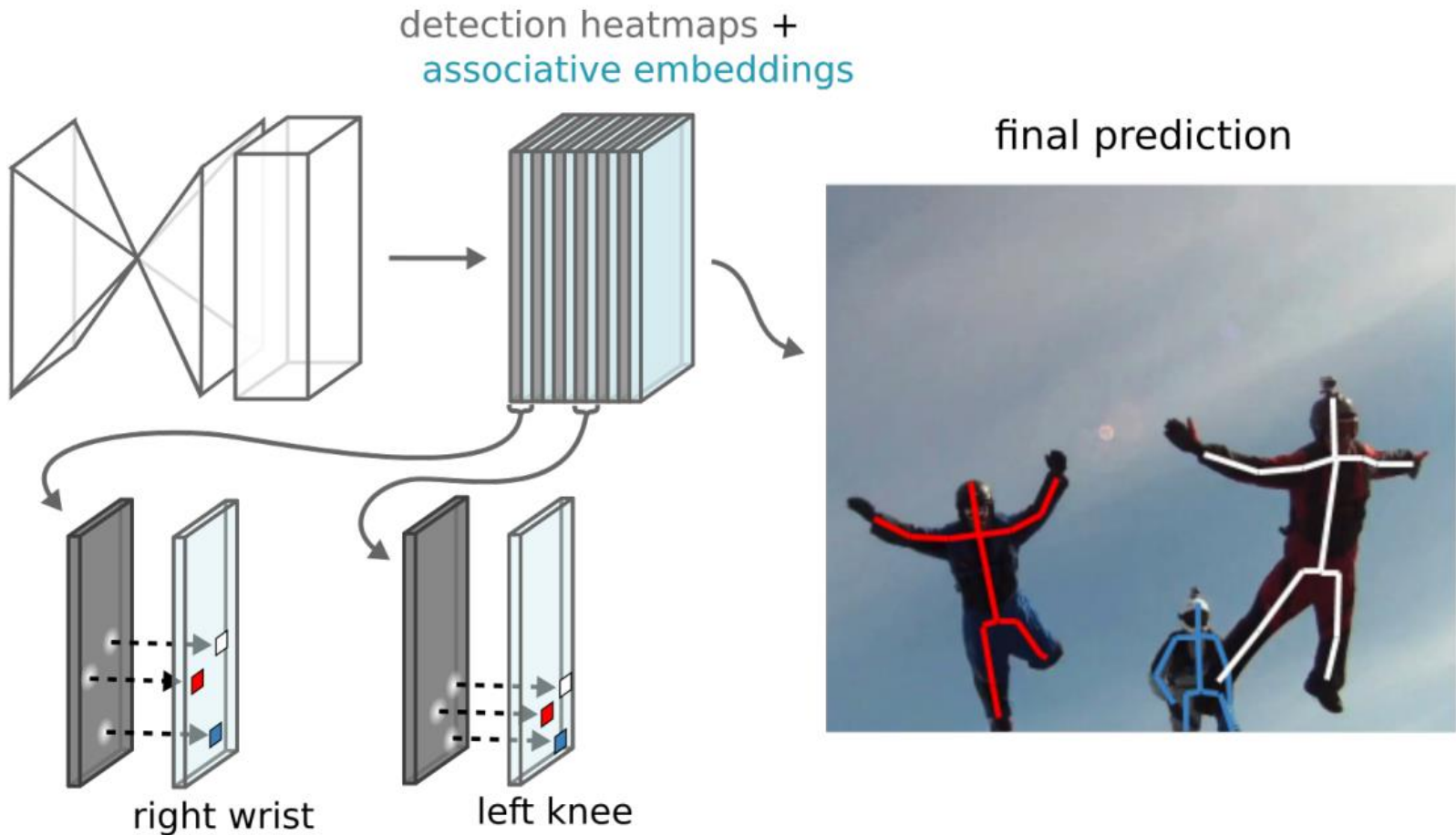
# Associative Embedding

- Idea: hourglass + use tag for grouping



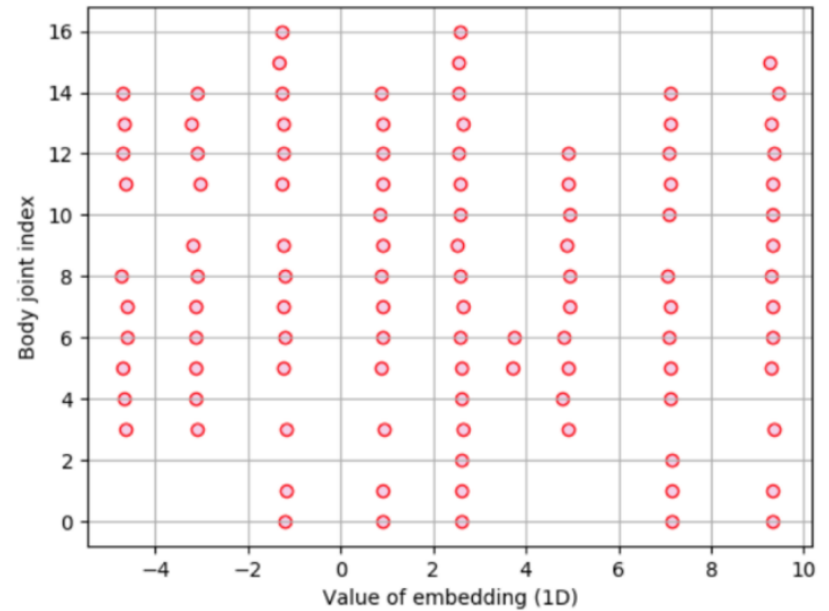
# Associative Embedding

- The algorithmic pipeline of AE



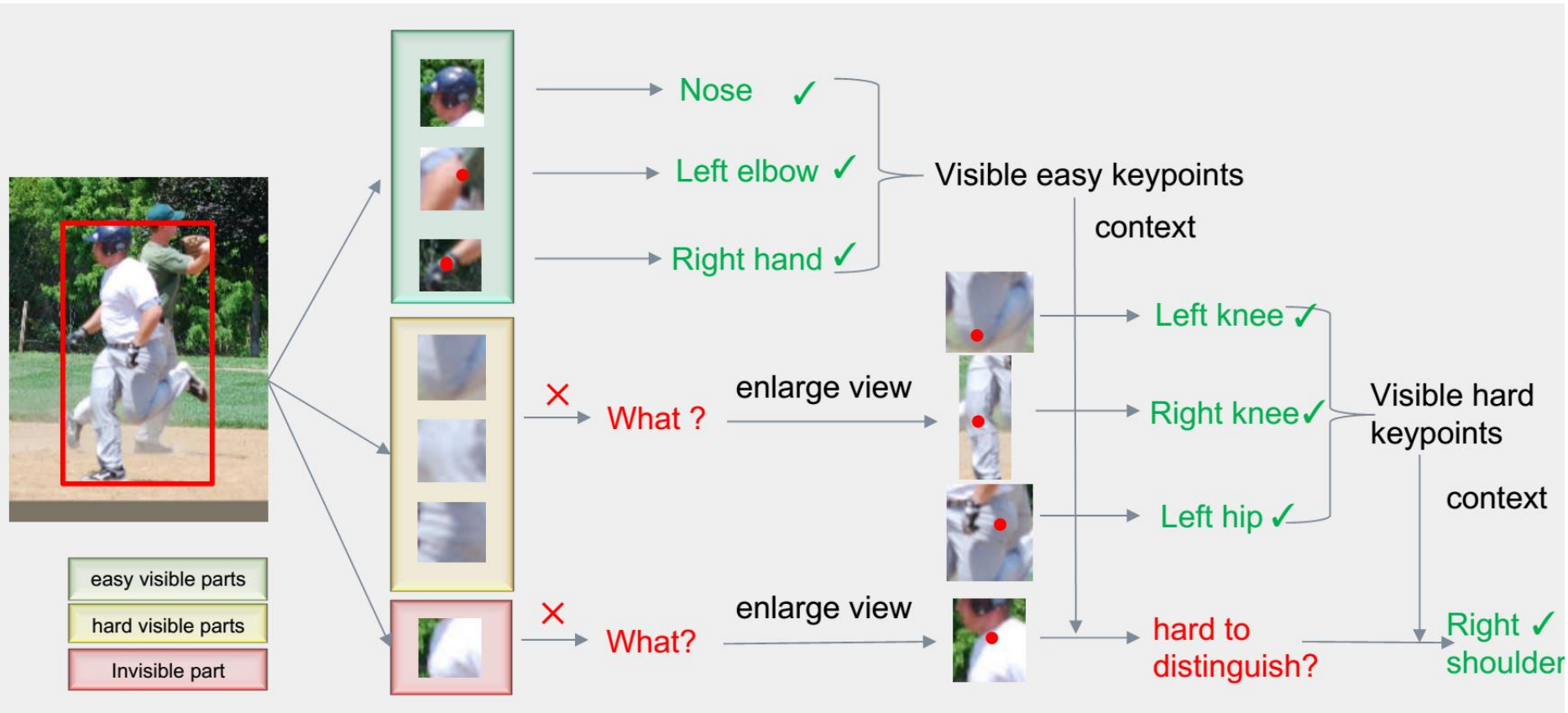
# Associative Embedding

- From 1-D tag to joint grouping



# Top-Down Method: Cascaded Pyramid Network

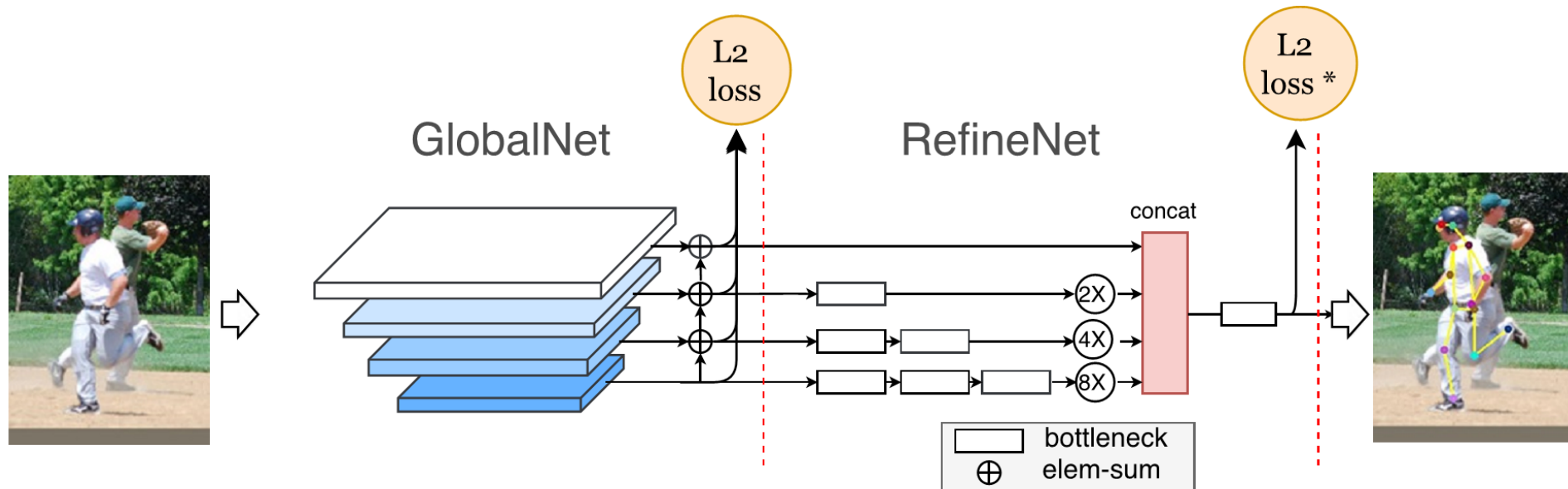
- MS COCO keypoint challenge 2017 winner





# Top-Down Method: Cascaded Pyramid Network

- MS COCO keypoint challenge 2017 winner





# Top-Down Method: Cascaded Pyramid Network

- Some experimental insights

**spatial resolution / cascade**

| Models            | Input Size       | FLOPs | AP(OKS) |
|-------------------|------------------|-------|---------|
| 8-stage Hourglass | $256 \times 192$ | 19.5G | 66.9    |
| 8-stage Hourglass | $256 \times 256$ | 25.9G | 67.1    |
| CPN* (ResNet-50)  | $256 \times 192$ | 6.2G  | 68.6    |
| CPN (ResNet-50)   | $256 \times 192$ | 6.2G  | 69.4    |
| CPN* (ResNet-50)  | $384 \times 288$ | 13.9G | 70.6    |
| CPN (ResNet-50)   | $384 \times 288$ | 13.9G | 71.6    |

**Is stacked hourglass useful?**

| Model                                 | FLOPs | Pckh-0.5 (MPI val) | AP@OKS0.75 (COCO val) |
|---------------------------------------|-------|--------------------|-----------------------|
| 1-stage hourglass( $256 \times 192$ ) | 3.9G  | 0.893              | 0.663                 |
| 2-stage hourglass( $256 \times 192$ ) | 6.1G  | 0.921              | 0.755                 |
| 3-stage hourglass( $256 \times 192$ ) | 8.3G  | 0.924              | 0.754                 |
| 4-stage hourglass( $256 \times 192$ ) | 10.5G | 0.924              | 0.752                 |

# SimpleBaseline and HRNet

- Resolution is crucial for point-based estimation

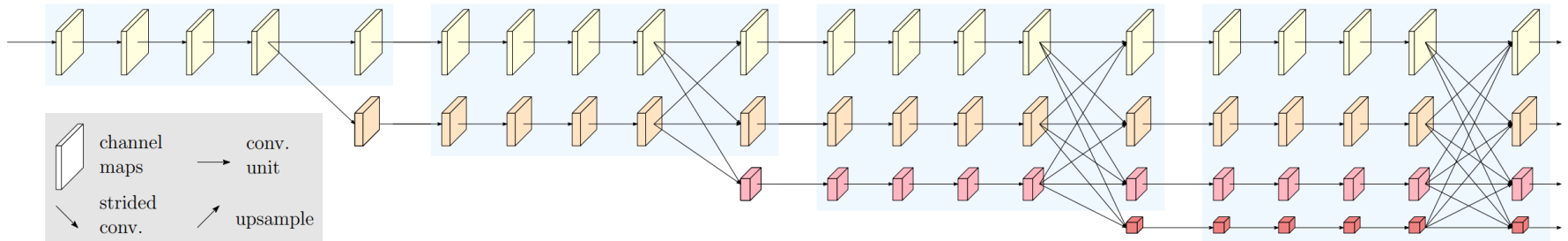
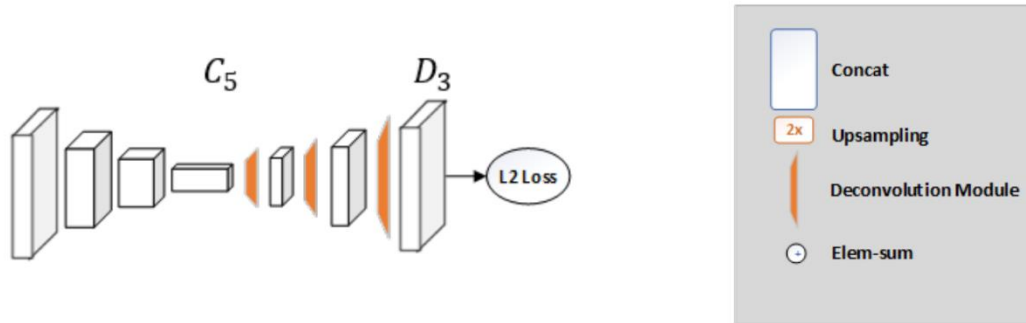


Fig. 2. An example of a high-resolution network. Only the main body is illustrated, and the stem (two stride-2  $3 \times 3$  convolutions) is not included. There are four stages. The 1st stage consists of high-resolution convolutions. The 2nd (3rd, 4th) stage repeats two-resolution (three-resolution, four-resolution) blocks. The detail is given in Section 3.

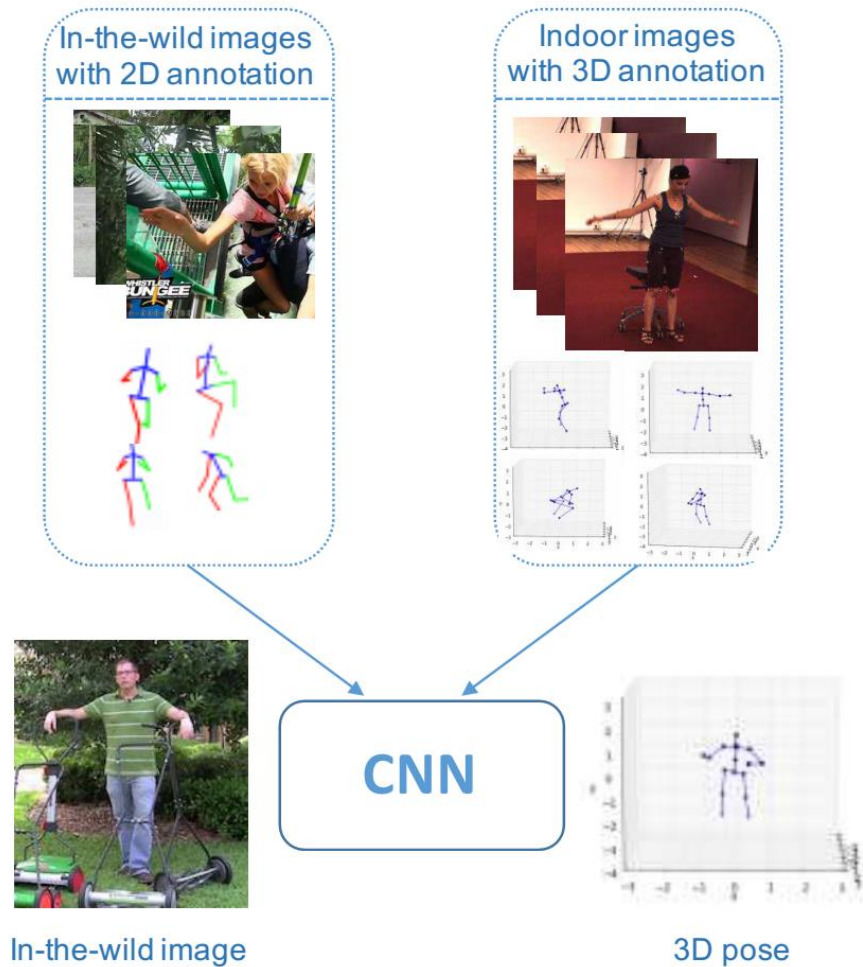
# 3D Pose Estimation

- Motion capture based: Human3.6M
- Multi-camera system based: 3DHP, CMU Panoptic Studio
- Synthetic data: Surreal

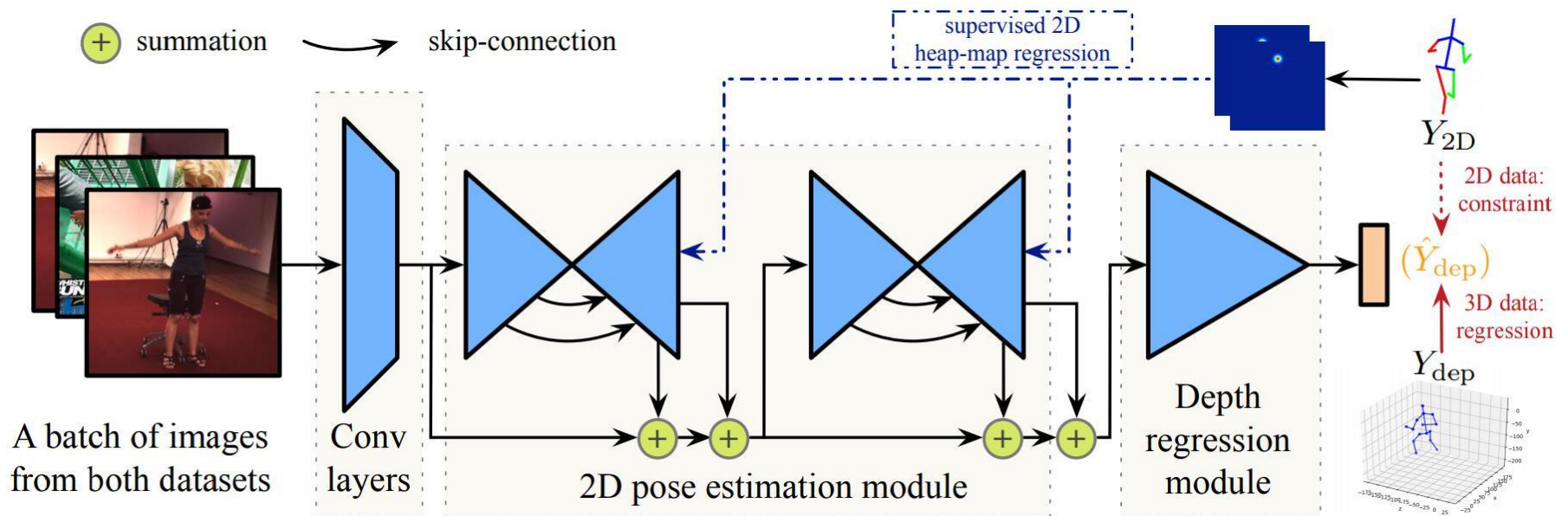


# 3D Pose Estimation

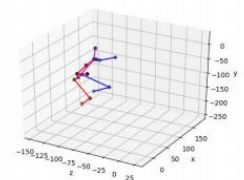
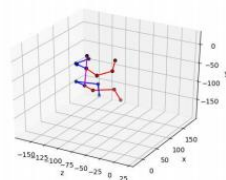
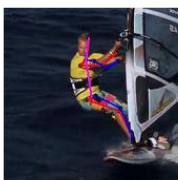
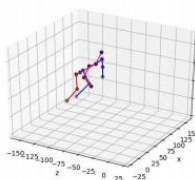
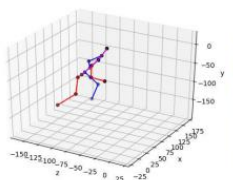
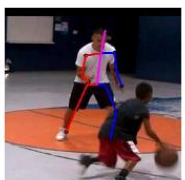
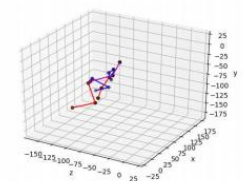
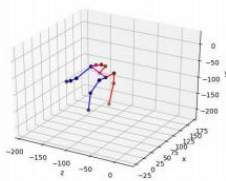
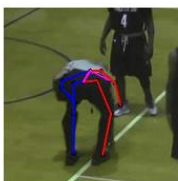
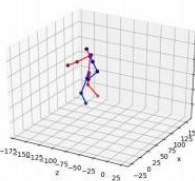
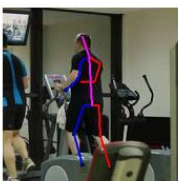
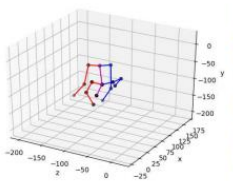
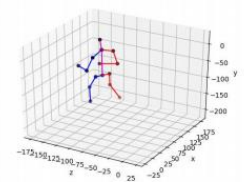
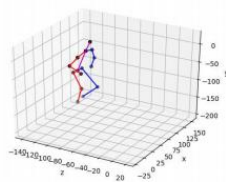
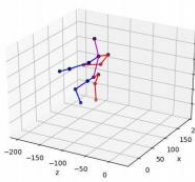
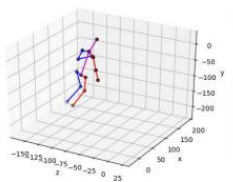
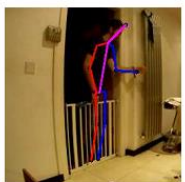
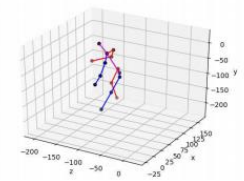
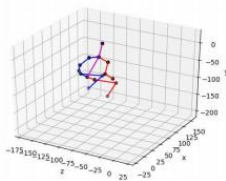
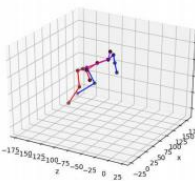
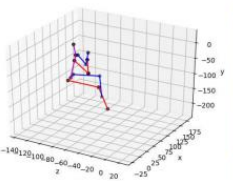
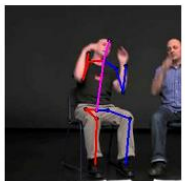
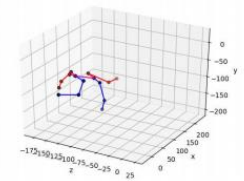
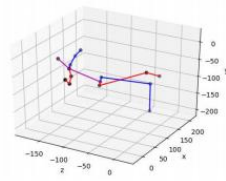
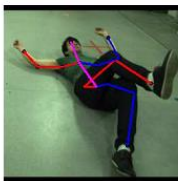
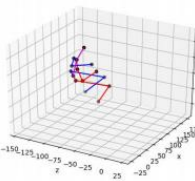
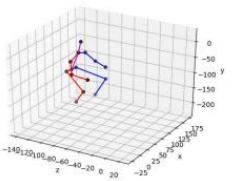
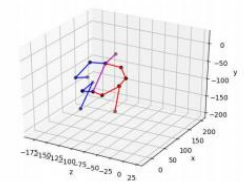
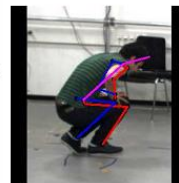
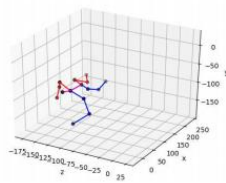
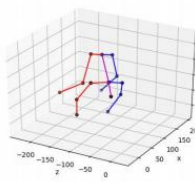
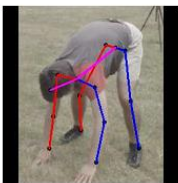
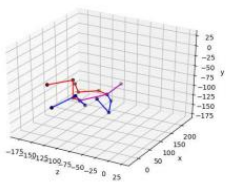
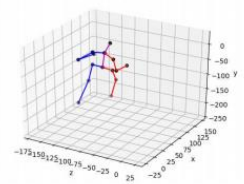
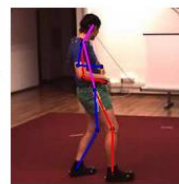
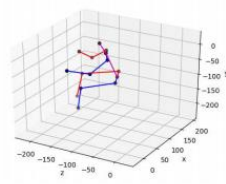
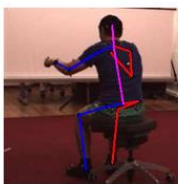
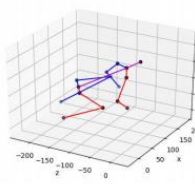
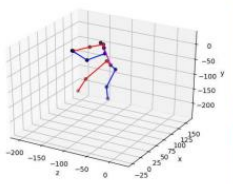
- The gap between 2D and 3D pose data



- Constrain 2D pose with geometry



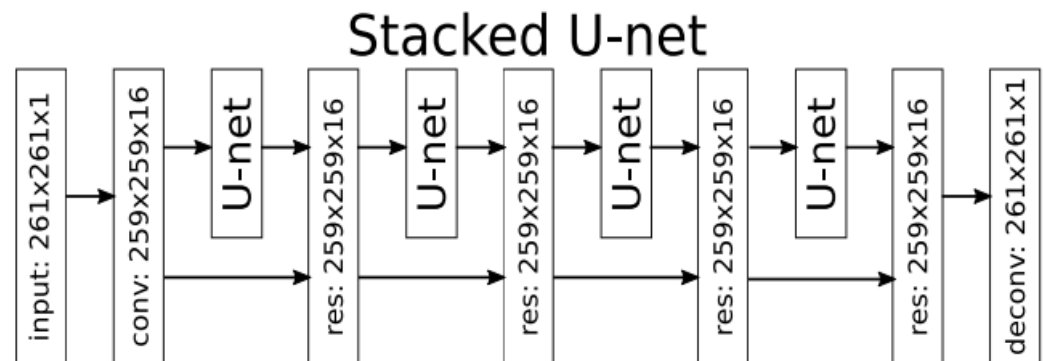
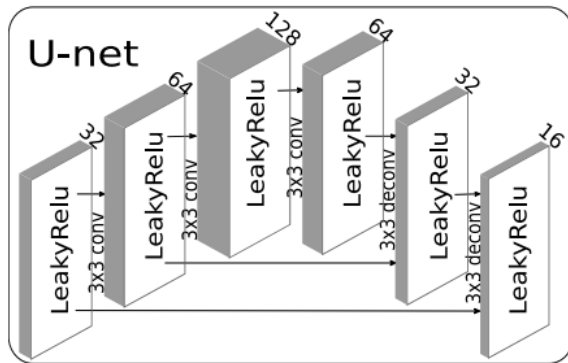






# Stacked U-Net is Everywhere

- Relativistic hydrodynamics is a powerful tool to simulate the evolution of the quark gluon plasma (QGP) in relativistic heavy ion collisions.
- Predict new states from some initial states
- Use stacked U-net as a general function-approximating tool



# Stacked U-Net is Everywhere

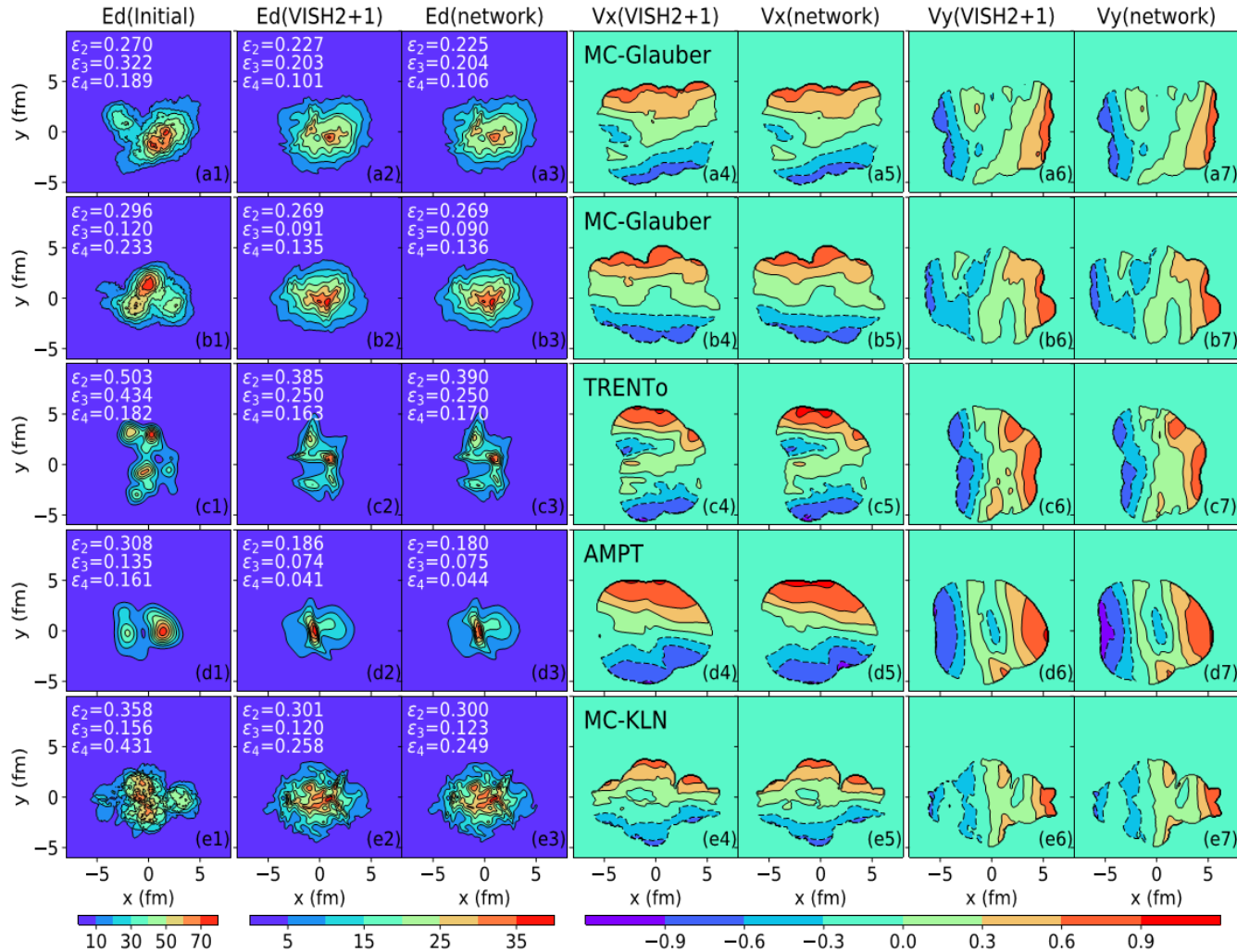


FIG. 2: Energy density and flow velocity profiles at  $\tau - \tau_0 = 2.0$  fm/c, calculated from VISH2+1 and predicted by the network for five test cases with initial profiles generated from MC-Glauber, MC-KLN, AMPT and TRENTo.

**Questions?**